

Flutter từ cơ bản đến nâng cao — Tập 1: Cơ bản

Cài đặt và chạy trên iPhone, Flutter cho Angular/React Native developer, Dart, widget, layout, theme, danh sách, form, go_router

Flutter 3.47.5 · Dart 3.13.4 · provider 6.1.5+1 · go_router 18.0.2 · sqflite 2.4.4

Bản dựng: 2026-09-30 — Markdown là nguồn chính (books/flutter/vol1-co-ban/)

Tham khảo docs.flutter.dev (CC BY 3.0) và dart.dev (CC BY 4.0) — xem Nguồn tham khảo cuối mỗi chương

Kiểm tra dấu tiếng Việt: ă ằ ẳ ẵ ơ ư ơ đ ư đ — Đường ướn, trắng khuyết, Ưu tiên

Mục lục

Chương 1 — Cài đặt, app đầu tiên và chạy trên iPhone

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 2 — Flutter cho Angular (và React Native) developer

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 3 — Dart cho TypeScript developer

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 4 — Widget: Stateless, Stateful, vòng đời và BuildContext

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 5 — Layout, styling, theming và adaptive layout

Mục tiêu

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[Chương 6 — Danh sách và cuộn: ListView, GridView, Dismissible, slivers](#)

[Mục tiêu](#)

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[Chương 7 — Form và nhập liệu](#)

[Mục tiêu](#)

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[Chương 8 — Điều hướng: Navigator và go_router](#)

[Mục tiêu](#)

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[Chương 9 — App mẫu Tập 1: "Việc Cần Làm"](#)

[Mục tiêu](#)

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

STACK — Phiên bản và thư viện dùng cho cả bộ sách Flutter

1. Tóm tắt nhanh (TL;DR)
2. Flutter và Dart mới nhất
3. Docs chính thức khuyên gì? (state, DI, routing, dữ liệu, test)
4. Chạy app Flutter trên iPhone — nói thật
5. Thư viện cho từng tập (bảng cho Task 9)
6. Công cụ trong sandbox (để kiểm tra sách)
7. Bản quyền nội dung tham khảo

Nguồn tham khảo (Sources)

GLOSSARY — Thuật ngữ Flutter (Anh → Việt)

Chương 1 — Cài đặt, app đầu tiên và chạy trên iPhone

Mục tiêu

- Cài Flutter SDK (Dart đi kèm), kiểm tra bằng `flutter doctor`.
- Tạo và chạy app đầu tiên; dùng **hot reload**.
- Hiểu cấu trúc một dự án Flutter (`lib/`, `test/`, `pubspec.yaml`, `ios/`, `android/`, `web/`).
- Biết **thật rõ** các cách chạy app trên iPhone: Mac + Xcode, hoặc bản web trên Safari.
- Biết DevTools là gì.

Giải thích đơn giản

Flutter giống **Angular CLI + một engine vẽ**: bạn viết Dart, lệnh `flutter` biên dịch và chạy trên thiết bị. Khác React Native + Expo: **không có app "Expo Go"** để quét QR. Muốn chạy trên iPhone thật, app phải được **build bằng Xcode trên máy Mac** rồi cài qua cáp (hoặc qua TestFlight). Nếu chưa có Mac, bạn vẫn học được toàn bộ sách bằng cách chạy trên **Chrome** (máy tính) và mở **bản web trên Safari** của iPhone.

Việc	Angular	React Native (Expo)	Flutter
Tạo dự án	<code>ng new</code>	<code>npx create-expo-app</code>	<code>flutter create</code>
Chạy dev	<code>ng serve</code>	<code>npx expo start</code>	<code>flutter run -d chrome</code> / <code>flutter run</code>
Cập nhật khi lưu	HMR	Fast Refresh	Hot reload (<code>r</code>), hot restart (<code>R</code>)
Cài package	<code>npm i x</code>	<code>npx expo install x</code>	<code>flutter pub add x</code>
Kiểm tra môi trường	—	<code>npx expo-doctor</code>	<code>flutter doctor</code>

Ví dụ

Bước 1 — Cài Flutter

Docs chính thức (Learning Pathway, bước "Quick install") hướng dẫn cài bằng VS Code + extension Flutter, hoặc tải SDK thủ công. Cài Flutter là có Dart. Sách dùng **Flutter 3.47.5** (stable, 2026-09-18) — xem [STACK.md](#).

Trên macOS/Linux, cách thủ công (sandbox của sách làm đúng như vậy):

```
# Linux (bản đã dùng để kiểm tra sách):
curl -sS -o /tmp/f.tar.xz
https://storage.googleapis.com/flutter_infra_release/releases/stable/linux/flutter_linux_3.47.5-stable.tar.xz
tar xf /tmp/f.tar.xz -C /opt
export PATH=/opt/flutter/bin:$PATH
flutter --version
```

Kết quả thật (sandbox, 2026-09-30):

```
Flutter 3.47.5 • channel stable • https://github.com/flutter/flutter.git
Framework • revision 6a19cca564 (13 days ago) • 2026-09-17 14:13:22 -0400
Engine • hash ab598368592da0064197e2bc15c7f5b0a2c6bb1f (revision af7e796e16) (13 days ago) • 2026-09-16 18:35:09.000Z
Tools • Dart 3.13.4 • DevTools 2.60.0
```

Trên Mac, nên dùng VS Code → Command Palette → "Flutter: New Project" như docs; extension sẽ tải SDK giúp bạn.

Bước 2 — flutter doctor

`flutter doctor` liệt kê cái gì còn thiếu. Kết quả thật trong sandbox (không có Android SDK, Chrome, Xcode):

```
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.5, on Ubuntu 24.04.4 LTS 6.18.44-fc-v50, locale en_US)
[x] Android toolchain - develop for Android devices
    x Unable to locate Android SDK.
[x] Chrome - develop for the web (Cannot find Chrome executable at google-chrome)
    ! Cannot find Chrome. Try setting CHROME_EXECUTABLE to a Chrome executable.
[x] Linux toolchain - develop for Linux desktop
[✓] Connected device (1 available)
[✓] Network resources

! Doctor found issues in 3 categories.
```

(Đã rút gọn vài dòng gợi ý.) Trên Mac của bạn, mục **Xcode** sẽ xuất hiện; cần nó xanh (✓) thì mới chạy được trên iPhone. Không cần mọi mục đều xanh: chỉ cần nền tảng bạn định chạy.

Bước 3 — Tạo và chạy app

```
flutter create --org dev.nobin --platforms=ios,android,web hello_flutter
cd hello_flutter
flutter run -d chrome          # mở Chrome
# khi app đang chạy: sửa code, lưu, bấm r (hot reload) hoặc R (hot restart), q để
thoát
```

--org đặt tiền tố bundle id cho iOS/Android (`dev.nobin.hello_flutter`). Dự án của sách tạo đúng bằng lệnh này (xem `examples/`).

App đầu tiên của sách — bộ đếm tiếng Việt (`examples/lib/chapters/ch01/hello_counter.dart`):

```
class HelloCounter extends StatefulWidget {
  const HelloCounter({super.key, this.step = 1});

  final int step;

  @override
  State<HelloCounter> createState() => _HelloCounterState();
}

class _HelloCounterState extends State<HelloCounter> {
  int _count = 0;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Text('Xin chào Flutter!', style:
Theme.of(context).textTheme.headlineMedium),
          const SizedBox(height: 12),
          Text('Bạn đã bấm $_count lần'),
          const SizedBox(height: 12),
          FilledButton.icon(
            onPressed: () => setState(() => _count += widget.step),
            icon: const Icon(Icons.add),
            label: const Text('Bấm tôi'),
          ),
        ],
      ),
    );
  }
}
```

Đọc nhanh (Chương 4 giải thích kỹ):

- `StatefulWidget` = widget có state. State nằm ở class `_HelloCounterState` (dấu `_` = private trong file).
- `setState(() => ...)` báo Flutter: "state đổi rồi, gọi lại `build`".
- `widget.step` = đọc tham số của widget (giống `this.step` của một `@Input`).

Điểm vào app (`examples/lib/main.dart`):

```
void main() {
  runApp(const TodoApp());
}
```

Bước 4 — Test đầu tiên

```
testWidgets('HelloCounter tăng theo step khi bấm', (tester) async {  
  await pumpApp(tester, const HelloCounter(step: 2));  
  expect(find.text('Bạn đã bấm 0 lần'), findsOneWidget);  
  await tester.tap(find.text('Bấm tôi'));  
  await tester.pump(); // build lại sau setState  
  expect(find.text('Bạn đã bấm 2 lần'), findsOneWidget);  
});
```

`pumpApp` là hàm nhỏ của sách (`examples/test/helpers.dart`) bọc widget trong `MaterialApp` + `Scaffold`. Kết quả thật (2026-09-30, `flutter test --reporter expanded test/chapters/ch01_test.dart`):

```
00:00 +0: Chương 1 – app đầu tiên HelloCounter tăng theo step khi bấm  
00:00 +1: Chương 1 – app đầu tiên Bài tập: nút Đặt lại bị vô hiệu khi = 0 và đưa về 0  
00:00 +2: All tests passed!
```

Bước 5 — Chạy app mẫu của sách

```
cd books/flutter/vol1-co-ban/examples  
flutter pub get  
flutter analyze           # kết quả thật: No issues found!  
flutter test              # kết quả thật: 58 test, All tests passed!  
flutter run -d chrome
```

Tab **Lab** mở ví dụ của từng chương. Toàn bộ kết quả kiểm tra: [../logs/check-all.txt](#).

Đi sâu

Chạy trên iPhone: 3 con đường thật

A. Mac + Xcode + cáp (cách chính thức). Theo docs `platform-integration/ios/setup`:

1. Cài Xcode; chạy `sudo sh -c 'xcode-select -s /Applications/Xcode.app/Contents/Developer && xcodebuild -runFirstLaunch'`, `sudo xcodebuild -license`, `xcodebuild -downloadPlatform iOS`.
2. Cắm iPhone vào Mac, bấm **Trust** (Tin cậy).
3. Bật **Developer Mode**: Settings → Privacy & Security → Developer Mode → On → khởi động lại → Turn On. (Nếu không thấy mục này: cắm máy, mở Xcode hoặc chạy `flutter run` một lần.)
4. Tạo chứng chỉ ký bằng Apple ID. Docs viết: tài khoản developer **cá nhân, miễn phí** dùng được khi chỉ *test* trên máy của mình; muốn lên App Store thì cần tài khoản trả phí.
5. Trên iPhone: Settings → VPN & Device Management → chọn chứng chỉ developer → **Trust**.

6. `flutter run` (chọn iPhone). Hot reload chạy được trên máy thật.

Từ Flutter 3.44, phụ thuộc native iOS được quản lý bằng **Swift Package Manager** (mặc định); CocoaPods chỉ còn dùng cho plugin chưa hỗ trợ SwiftPM (docs `swift-package-manager/for-app-developers`).

UNVERIFIED: bản cài bằng Apple ID miễn phí hết hạn sau 7 ngày (cộng đồng hay nói; chưa mở được trang Apple xác nhận).

B. Bản web trên Safari (không cần Mac). Flutter web chạy trên Safari mobile (docs `web/faq`):

```
cd books/flutter/vol1-co-ban/examples
flutter build web --no-web-resources-cdn --release
cd build/web && python3 -m http.server 8080 --bind 0.0.0.0
# iPhone cùng Wi-Fi: Safari → http://<IP-máy-tính>:8080 → Chia sẻ → "Thêm vào MH chính"
```

- Không dùng `--wasm` cho iPhone: docs `web/wasm` nói bản Wasm không chạy trên trình duyệt nào của iOS.
- Web **không phải app native**: không có thông báo cục bộ theo lịch, giọng đọc TTS dùng Web Speech của Safari.
- Trong sandbox, sách đã kiểm tra bản web bằng Chromium headless (script `scripts/web-smoke.mjs`); trên Safari iOS: **NOT RUN**.

C. TestFlight. Build trên Mac (hoặc Mac trên cloud CI), upload lên App Store Connect, cài bằng app TestFlight. Cần **Apple Developer Program (\$99/năm** — trang `developer.apple.com/programs`, mở 2026-09-30). Tập 3 Ch.5–6.

Cấu trúc dự án

```
examples/
pubspec.yaml      # tên app, phiên bản, phụ thuộc (≈ package.json)
pubspec.lock      # khóa phiên bản (≈ package-lock.json) – commit với app
analysis_options.yaml # luật lint (≈ eslint config)
lib/main.dart     # điểm vào
lib/...           # code Dart của bạn
test/             # test (chạy bằng flutter test)
ios/ android/ web/ # dự án native / web do flutter create sinh ra
```

Hot reload hoạt động thế nào?

Ở chế độ debug, Dart chạy bằng **JIT** (biên dịch lúc chạy) nên có thể nạp code mới vào app đang chạy mà giữ state. Bản release dùng **AOT** (biên dịch trước ra mã máy) nên nhanh hơn nhưng không hot reload. Docs `testing/build-modes` mô tả 3 chế độ: debug, profile, release.

DevTools

`flutter run` in ra một đường link DevTools. DevTools có: **Widget Inspector** (xem cây widget, giống Angular DevTools), **Performance**, **CPU profiler**, **Memory**, **Network**, **Logging**. Learning Pathway có một bài riêng về DevTools; sách dùng lại ở Tập 3 (Performance).

Lỗi và bẫy thường gặp

- **Tưởng có "Expo Go cho Flutter"** — không có. Cần Mac + Xcode cho app native trên iPhone.
- `flutter doctor` **đỏ hết rồi hoảng**: chỉ cần nền tảng bạn dùng xanh.
- **Chạy với quyền root** in cảnh báo "Woah! You appear to be trying to run flutter as root" — vô hại trong sandbox, nhưng trên máy cá nhân đừng dùng `sudo flutter`.
- **Quên `flutter pub get`** sau khi clone → lỗi "Target of URI doesn't exist".
- **Hot reload không áp dụng** khi sửa `main()` hoặc `initState` — dùng hot restart (R).
- **Web trắng trang khi mạng bị chặn**: mặc định Flutter web tải CanvasKit từ CDN; dùng `--no-web-resources-cdn`.

Tóm tắt

- Cài Flutter = có Dart. Kiểm tra bằng `flutter doctor`. Sách ghim Flutter 3.47.5 / Dart 3.13.4.
- `flutter create` → `flutter run -d chrome` → sửa code → `r` hot reload.
- iPhone: Mac + Xcode (Apple ID miễn phí để test), hoặc bản web trên Safari, hoặc TestFlight (trả phí).
- Mọi code của sách nằm trong `examples/` và được `flutter analyze` + `flutter test` kiểm tra.

Bài tập (có lời giải)

Bài 1. Viết widget `CounterWithReset`: nút "+1" và nút "Đặt lại". Nút "Đặt lại" bị **vô hiệu** khi bộ đếm = 0.

▼ Lời giải

`examples/lib/chapters/ch01/exercise_solution.dart` (đoạn chính):

```

Row(
  mainAxisAlignment: MainAxisAlignment.min,
  children: [
    FilledButton(onPressed: () => setState(() => _count++), child: const
Text('+1')),
    const SizedBox(width: 8),
    // onPressed = null → nút tự vô hiệu (disabled). Không cần thuộc tính
[disabled] riêng.
    OutlinedButton(
      onPressed: _count == 0 ? null : () => setState(() => _count = 0),
      child: const Text('Đặt lại'),
    ),
  ],
),

```

Test kiểm tra `onPressed` là `null` lúc đầu, bấm "+1" hai lần rồi "Đặt lại" về 0 (ch01_test.dart). Trong Angular bạn viết `[disabled]="count === 0"`; trong Flutter, "vô hiệu" = không có hàm xử lý.

Bài 2. Bạn không có Mac. Liệt kê các lệnh để Nobin mở app Tập 1 trên iPhone, và nói 2 giới hạn của cách đó.

▼ Lời giải

```

cd books/flutter/vol1-co-ban/examples
flutter build web --no-web-resources-cdn --release
cd build/web && python3 -m http.server 8080 --bind 0.0.0.0
# iPhone cùng Wi-Fi → Safari → http://<IP máy tính>:8080

```

Giới hạn: (1) đây là web, không phải app native — không có thông báo cục bộ theo lịch, không truy cập đủ API thiết bị; (2) không dùng được `--wasm` trên iOS. Lệnh build web đã chạy thật trong sandbox (logs/check-all.txt: "✓ Built build/web" và "WEB SMOKE OK"); mở trên Safari iOS: NOT RUN.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (docs.flutter.dev bị chặn trong sandbox; đọc file nguồn ở commit ab59c61):

- Learning pathway — <https://docs.flutter.dev/learn/pathway> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/index.md>
- Quick install / test drive — <https://docs.flutter.dev/install/quick> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/install/quick.md>

- Tutorial: Create a Flutter app — <https://docs.flutter.dev/learn/pathway/tutorial/create-an-app> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/create-an-app.md>
- Tutorial: DevTools — <https://docs.flutter.dev/learn/pathway/tutorial/devtools> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/devtools.md>
- Set up iOS development — <https://docs.flutter.dev/platform-integration/ios/setup> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/platform-integration/ios/setup.md>
- Swift Package Manager for app developers — <https://docs.flutter.dev/packages-and-plugins/swift-package-manager/for-app-developers> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/packages-and-plugins/swift-package-manager/for-app-developers.md>
- Building a web application — <https://docs.flutter.dev/platform-integration/web/building> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/platform-integration/web/building.md>
- Web FAQ — <https://docs.flutter.dev/platform-integration/web/faq> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/platform-integration/web/faq.md>
- Wasm — <https://docs.flutter.dev/platform-integration/web/wasm> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/platform-integration/web/wasm.md>
- Build modes — <https://docs.flutter.dev/testing/build-modes> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/testing/build-modes.md>
- Flutter releases (Linux): https://storage.googleapis.com/flutter_infra_release/releases/releases_linux.json
- Apple Developer Program: <https://developer.apple.com/programs/>

Chương 2 — Flutter cho Angular (và React Native) developer

Mục tiêu

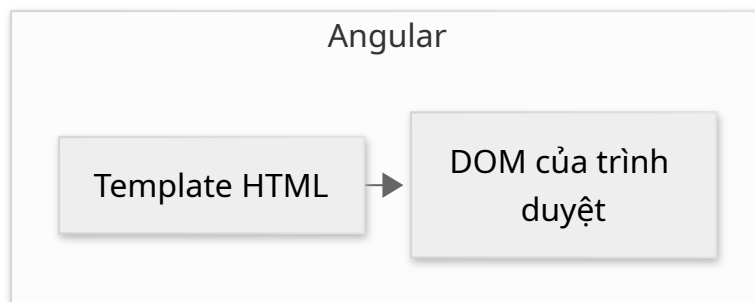
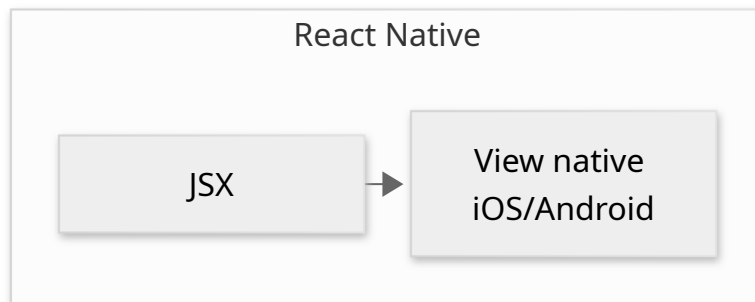
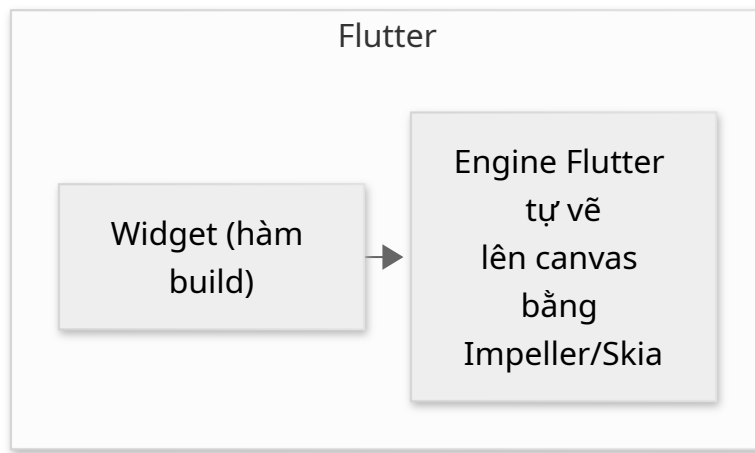
Bạn đã giỏi Angular và đã học React Native (RN). Chương này giúp bạn **chuyển kiến thức** sang Flutter nhanh:

- Đối chiếu từng khái niệm: component ↔ widget, template ↔ hàm `build`, service/DI, RxJS/Signals ↔ Stream/ValueNotifier/ChangeNotifier, routing, forms, testing.
- Hiểu điểm khác lớn nhất: Flutter **tự vẽ mọi pixel** (không dùng view native như RN, không dùng DOM như Angular) và UI là **khai báo (declarative)**: `UI = f(state)`.
- Viết 4 ví dụ nhỏ có test: @Input/@Output, Signals, DI, `debounceTime`.

Giải thích đơn giản

Docs chính thức ("Introduction to declarative UI") giải thích: với kiểu **mệnh lệnh (imperative)**, bạn tìm một view rồi gọi hàm để sửa nó. Với kiểu **khai báo (declarative)** của Flutter, bạn chỉ **mô tả giao diện ứng với state hiện tại**; khi state đổi, Flutter gọi lại hàm `build` và tự tính phần cần vẽ lại.

Angular cũng khai báo (template), React/RN cũng vậy (JSX). Khác biệt nằm ở **chỗ vẽ**:



Hệ quả thực tế:

- App Flutter trông **giống hệt nhau** trên iPhone, Android, web (vì tự vẽ). Muốn "đúng chất iOS" thì dùng widget Cupertino hoặc `.adaptive`.
- Không có CSS, không có HTML. **Mọi thứ là widget**: cả khoảng cách (`Padding`), căn giữa (`Center`), bố cục (`Row`, `Column`).
- Code là **Dart** (không phải TypeScript) — Chương 3 dạy Dart cho người biết TypeScript.

Ví dụ

Bảng đối chiếu chính

Angular	React Native	Flutter (sách dùng)	Ghi chú
@Component class + template	Hàm component + JSX	Class StatelessWidget / StatefulWidget với hàm build()	Template nằm ngay trong Dart
<div>	<View>	Container, Padding, SizedBox, Row, Column	Mỗi widget làm một việc
, <p>	<Text>	Text	
<button (click)>	<Pressable onPress>	FilledButton(onPressed:), IconButton, InkWell	onPressed: null = nút bị vô hiệu
<input [(ngModel)]>	<TextInput value onChangeText>	TextField + TextEditingController	Chương 7
*ngFor / @for	.map() / FlatList	for trong list con, ListView.builder	Chương 6
*ngIf / @if	{cond && <A/>}	if (cond) Widget() trong list con, hoặc cond ? A() : B()	"collection if" của Dart
SCSS, [class.x]	StyleSheet	Tham số của widget (style: , padding:) + ThemeData	Chương 5
@Input() / input()	props	Tham số constructor (final field)	Chỉ đọc
@Output() / output()	callback prop	Callback (ValueChanged<T> , VoidCallback)	
ngOnInit	useEffect(..., [])	initState()	Trong State của StatefulWidget
ngOnChanges	useEffect(..., [prop])	didUpdateWidget(old)	
ngOnDestroy	cleanup của useEffect	dispose()	Hủy Timer, controller, subscription
signal()	useState	setState (cục bộ), ValueNotifier	
computed()	useMemo	getter / tính trong build; Computed tự viết (bài tập)	
Service @Injectable có state	Zustand store	Class extends ChangeNotifier (ViewModel)	Tập 2
DI: providers, inject()	Context	InheritedWidget; package provider (docs khuyên)	Tập 2 dùng provider
RxJS Observable	Promise + hook	Future (1 giá trị), Stream (nhiều giá trị)	Dart có Stream sẵn trong ngôn ngữ

Angular	React Native	Flutter (sách dùng)	Ghi chú
async pipe	—	FutureBuilder, StreamBuilder	Tập 2
HttpClient	fetch	package http	Tập 2
Angular Router	Expo Router	go_router (docs khuyên)	Chương 8
routerLink, router.navigate	<Link>, router.push	context.go('/path'), context.push()	
ActivatedRoute.paramMap	useLocalSearchParams	state.pathParameters['id']	
Guard canActivate	<Redirect>	redirect: của GoRouter	Chương 8
Reactive Forms + Validators	useState + validate	Form + TextFormField validator:	Chương 7
TestBed + Jasmine/Jest	Jest + RNTL	flutter_test: testWidgets, find, tester.tap	Có sẵn trong SDK
ng serve (HMR)	npx expo start (Fast Refresh)	flutter run (hot reload — phím r)	
ng build	eas build	flutter build ios / apk / web	Tập 3
package.json + npm	package.json + npm	pubspec.yaml + flutter pub (pub.dev)	

Ví dụ 1 — @Input/@Output → tham số + callback

Angular:

```
@Component({ selector: 'app-rating', template: `...` })
export class RatingComponent {
  @Input() value = 0;
  @Output() valueChange = new EventEmitter<number>();
}
```

Flutter (examples/lib/chapters/ch02/rating_stars.dart):

```

class RatingStars extends StatelessWidget {
  const RatingStars({super.key, required this.value, required this.onChanged, this.max
= 5});

  final int value;
  final int max;
  final ValueChanged<int> onChanged;

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        for (var n = 1; n <= max; n++)
          IconButton(
            tooltip: '$n sao',
            isSelected: n <= value,
            icon: const Icon(Icons.star_border),
            selectedIcon: const Icon(Icons.star, color: Colors.amber),
            onPressed: () => onChanged(n),
          ),
      ],
    );
  }
}

```

Cha giữ state và truyền xuống: `RatingStars(value: _rating, onChanged: (v) => setState(() => _rating = v))`. Đây chính là "banana in a box" `[(value)]` của Angular, nhưng viết tường minh — giống hệt RN (xem [RN Tập 1, Chương 2](#)).

Chú ý `for (...)` nằm **bên trong** danh sách `children` — đó là *collection for* của Dart, thay cho `@for`.

Ví dụ 2 — Signals → ValueNotifier

`ValueNotifier<T>` có sẵn trong Flutter: giữ một giá trị, báo cho người nghe khi giá trị **đổi** (so sánh bằng `==`). Rất giống `signal()`:

```

final ValueNotifier<int> _cartCount = ValueNotifier(0); // signal(0)
_cartCount.value++; // count.update(c => c + 1)

// Chỉ phần này build lại khi _cartCount đổi – giống signal được đọc trong template.
ValueListenableBuilder<int>(
  valueListenable: _cartCount,
  builder: (context, count, _) => Badge(
    label: Text('$count'),
    isLabelVisible: count > 0,
    child: const Icon(Icons.shopping_cart_outlined, size: 32),
  ),
),

```

(`examples/lib/chapters/ch02/cart_badge.dart`.) Khác Angular: Flutter **không tự theo dõi** signal nào được đọc (không có auto-tracking); bạn phải bọc bằng `ValueListenableBuilder` (hoặc `ListenableBuilder`). Nhớ `dispose()` notifier trong `dispose()` của `State`.

Ví dụ 3 — DI → `InheritedWidget`

```
abstract interface class GreetingService {
  String greet(String name);
}

class FriendlyGreeting implements GreetingService {
  const FriendlyGreeting();
  @override
  String greet(String name) => 'Chào $name!';
}

/// Giống `providers: [{ provide: GreetingService, useValue: ... }]` ở một nhánh của
cây.
class GreetingScope extends InheritedWidget {
  const GreetingScope({super.key, required this.service, required super.child});

  final GreetingService service;

  /// Giống `inject(GreetingService)`. Không có scope nào ở trên → dùng mặc định.
  static GreetingService of(BuildContext context) =>
    context.dependOnInheritedWidgetOfExactType<GreetingScope>()?.service ?? const
    FriendlyGreeting();

  @override
  bool updateShouldNotify(GreetingScope oldWidget) => service != oldWidget.service;
}
```

- `GreetingScope.of(context)` tìm lên trên cây widget — giống **hierarchical injector** của Angular.
- Bọc một nhánh bằng `GreetingScope(service: FormalGreeting(), child: ...)` → nhánh đó dùng bản khác.
- Trong test, bọc widget bằng scope chứa bản giả (fake) — giống `TestBed.overrideProvider`.
- Viết `InheritedWidget` bằng tay hơi dài. Docs chính thức khuyên dùng package `provider` cho DI (nó được xây trên `InheritedWidget`). Tập 2 dùng `provider`.

Ví dụ 4 — RxJS → Stream và Timer

Dart có `Stream` ngay trong thư viện chuẩn, với nhiều toán tử giống RxJS:

```
/// RxJS: `source.pipe(map(trim), filter(len >= 2), distinctUntilChanged())`
Stream<String> searchTerms(Stream<String> source) => source.map((s) =>
s.trim()).where((s) => s.length >= 2).distinct();
```

`debounceTime` không có sẵn, nhưng viết bằng `Timer` rất ngắn:

```
class Debouncer {
  Debouncer(this.delay);

  final Duration delay;
  Timer? _timer;

  void call(VoidCallback action) {
    _timer?.cancel();
    _timer = Timer(delay, action);
  }

  void dispose() => _timer?.cancel();
}
```

Ô tìm kiếm dùng: `onChanged: (text) => _debouncer(() => widget.onSearch(text))`, và gọi `_debouncer.dispose()` trong `dispose()` (giống `takeUntilDestroyed()`).

Kết quả test thật

Chạy ngày 2026-09-30, `flutter test --reporter expanded test/chapters/ch02_test.dart` (reporter "expanded" in dòng khi mỗi test **bắt đầu**; số +N là số test đã pass):

```
00:00 +0: Angular/React Native → Flutter RatingStars: dữ liệu đi xuống, callback đi lên (như @Input/@Output)
00:00 +1: Angular/React Native → Flutter ValueNotifier như signal: badge cập nhật khi bấm
00:01 +2: Angular/React Native → Flutter InheritedWidget thay implementation như providers: [...]
00:01 +3: Angular/React Native → Flutter SearchBox debounce 300ms (debounceTime)
00:01 +4: Angular/React Native → Flutter searchTerms: map / where / distinct giống pipe(map, filter, distinctUntilChanged)
00:01 +5: Angular/React Native → Flutter Bài 1: Computed tính lại khi nguồn đổi, chỉ báo khi kết quả đổi
00:01 +6: All tests passed!
```

Test debounce dùng **thời gian giả**: trong `testWidgets`, `await`

`tester.pump(Duration(milliseconds: 300))` cho thời gian trôi 300 ms ngay lập tức — giống `fakeAsync` + `tick(300)` của Angular.

Xem trong app: tab **Lab** → "Ch.2" (trên web đã chạy; trên iPhone: **NOT RUN**).

Đi sâu

Routing: Angular Router ↔ go_router

Angular	go_router
<code>const routes: Routes = [{ path: 'task/:id', component: TaskDetail }]</code>	<code>GoRoute(path: '/tasks/:id', builder: (context, state) => TaskDetailScreen(id: state.pathParameters['id']!))</code>
<code><router-outlet></code> + layout component	ShellRoute / StatefulShellRoute (tab giữ state)
<code>router.navigate(['/task', id])</code>	<code>context.go('/tasks/\$id')</code> (thay stack theo URL) / <code>context.push(...)</code> (chồng lên)
<code>canActivate guard</code>	<code>redirect: (context, state) => ...</code> (trả về URL mới hoặc null)
<code>queryParamsMap</code>	<code>state.uri.queryParameters</code>

Docs `ui/navigation`: không khuyên dùng *named routes* cho đa số app; khuyên dùng `go_router` để có deep link và URL trên web. Chương 8 đi chi tiết.

Forms: Reactive Forms ↔ Form + TextFormField

Angular	Flutter
<code>FormGroup</code>	<code>Form + GlobalKey<FormState></code>
<code>FormControl</code>	<code>TextFormField + TextEditingController</code>
<code>Validators.required</code>	<code>validator: (v) => v!.isEmpty ? 'Bắt buộc' : null</code> (trả về chuỗi lỗi hoặc null)
<code>form.markAllAsTouched();</code> <code>form.valid</code>	<code>formKey.currentState!.validate()</code>
<code>updateOn: 'change'</code>	<code>autovalidateMode: AutovalidateMode.onUserInteraction</code>

State: service có state ↔ ChangeNotifier (Tập 2)

Docs chính thức (Architecture recommendations) khuyên **MVVM**: View (widget) + ViewModel (`ChangeNotifier`), và DI bằng `provider`. Với Angular dev, ViewModel ≈ một service có state gắn với một màn hình; `notifyListeners()` ≈ `signal.set()` / `subject.next()`.

Testing: TestBed ↔ flutter_test

Angular	Flutter
TestBed.configureTestingModule + createComponent	await tester.pumpWidget(MaterialApp(home: X()))
fixture.detectChanges()	await tester.pump() (vẽ 1 frame) / pumpAndSettle() (chờ hết animation)
By.css('.btn')	find.text('Lưu'), find.byTooltip(...), find.byType(TextField)
fakeAsync + tick(300)	tester.pump(const Duration(milliseconds: 300))
HttpTestingController	MockClient của package http (Tập 2)

Tư duy khác biệt quan trọng

1. **Widget là bất biến (immutable)** và rất rẻ: `build` được gọi nhiều lần; đừng làm việc nặng trong `build`.
2. **Mọi thứ là widget**: padding, căn giữa, theme... đều là widget bọc ngoài (không có CSS cascade).
3. **Hai loại state** (docs `state-mgmt/ephemeral-vs-app`): state *cục bộ* (tab đang chọn, ô đang gõ) → `setState`; state *toàn app* (danh sách từ vựng, đăng nhập) → `ViewModel` + `provider`.
4. **Hot reload** giữ state khi sửa code; **hot restart** (R) chạy lại từ đầu.

Lỗi và bẫy thường gặp

- **Sửa field rồi quên `setState`**: UI không đổi. Luôn `setState(() => _x = ...)`.
- **Gọi `setState` sau `dispose`** (ví dụ sau `await`): kiểm tra `if (!mounted) return;` trước.
- **Quên `dispose()` controller/Timer/notifier** → rò rỉ bộ nhớ; `flutter_test` còn báo lỗi "A Timer is still pending".
- **Tạo đối tượng mới trong `updateShouldNotify/build` mỗi lần** → mọi widget con build lại vô ích.
- **Mang RxJS vào mọi thứ**: Dart đã có `Stream`; package `rxdart` có thêm toán tử nhưng thường không cần.
- **Tên trùng với thư viện test**: hàm tên `matches` trùng với matcher `matches` của `flutter_test` (Chương 7 gặp thật lỗi này và đổi tên thành `matchesField`).

Tóm tắt

- Widget = class có `build()`; template là code Dart. `@Input/@Output` = tham số + callback.
- Signals ≈ `ValueNotifier` + `ValueListenableBuilder`. Service có state ≈ `ChangeNotifier` (Tập 2).
- DI = `InheritedWidget`; thực tế dùng `provider` (docs khuyên). Routing = `go_router` (docs khuyên).

- RxJS $\approx$ Stream (có sẵn) + vài dòng Timer.
- Test bằng flutter_test có sẵn; thời gian giả bằng tester.pump(duration).

Bài tập (có lời giải)

Bài 1. Viết Computed<T> — tương đương computed() của Angular — nhận danh sách Listenable nguồn và một hàm tính. Khi một nguồn đổi thì tính lại; chỉ báo listener khi **kết quả** đổi. Nhớ gỡ listener khi dispose.

▼ Lời giải

examples/lib/chapters/ch02/exercise_solution.dart:

```
class Computed<T> extends ChangeNotifier implements ValueListenable<T> {
  Computed(this._sources, this._compute) : _value = _compute() {
    for (final s in _sources) {
      s.addListener(_recompute);
    }
  }

  final List<Listenable> _sources;
  final T Function() _compute;
  T _value;

  @override
  T get value => _value;

  void _recompute() {
    final next = _compute();
    if (next == _value) return;
    _value = next;
    notifyListeners();
  }

  @override
  void dispose() {
    for (final s in _sources) {
      s.removeListener(_recompute);
    }
    super.dispose();
  }
}
```

Test (ch02_test.dart): price = 100, qty = 2 → 200; qty.value = 3 → 300, listener được gọi đúng 1 lần. Vì Computed implement ValueListenable, bạn dùng nó thẳng với ValueListenableBuilder. Khác Angular: ta phải khai báo nguồn bằng tay (không có auto-tracking). Package signals trên pub.dev có auto-tracking giống Angular (so sánh ở Tập 2, Chương 1).

Bài 2. Viết lại service Angular sau theo kiểu Flutter, và nói cách thay bản giả trong test:

```
@Injectable({ providedIn: 'root' })
export class ClockService { now(): Date { return new Date(); } }
```

▼ Lời giải

Cách đơn giản nhất: **truyền hàm vào constructor** (DI bằng tham số). Widget `Clock` ở Chương 4 làm đúng như vậy:

```
class Clock extends StatefulWidget {
  const Clock({super.key, this.paused = false, this.now = DateTime.now});
  final DateTime Function() now;
  // ...
}
// Trong test: Clock(now: () => DateTime(2026, 1, 1, 8))
```

Nếu nhiều widget sâu bên dưới cùng cần, dùng mẫu `GreetingScope` ở Ví dụ 3 (một `ClockScope` `InheritedWidget`), hoặc `Provider<ClockService>` ở Tập 2. Cả hai cách đã được test: `ch04_test.dart` ("Clock cập nhật mỗi giây...") và `ch02_test.dart` ("InheritedWidget thay implementation...").

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (docs.flutter.dev bị chặn trong sandbox; đọc file nguồn ở commit `ab59c61`):

- Introduction to declarative UI — <https://docs.flutter.dev/flutter-for/declarative> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/flutter-for/declarative.md>
- Flutter for React Native developers — <https://docs.flutter.dev/flutter-for/react-native-devs> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/flutter-for/react-native-devs.md>
- Flutter for web developers — <https://docs.flutter.dev/flutter-for/web-devs> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/flutter-for/web-devs.md>
- Ephemeral vs app state — <https://docs.flutter.dev/data-and-backend/state-mgmt/ephemeral-vs-app> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/data-and-backend/state-mgmt/ephemeral-vs-app.md>
- Architecture recommendations — <https://docs.flutter.dev/app-architecture/recommendations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/recommendations.md>

[38028f581a5/sites/docs/src/content/app-architecture/recommendations.md](https://github.com/flutter/flutter/blob/master/docs/src/content/app-architecture/recommendations.md)

- Navigation and routing — <https://docs.flutter.dev/ui/navigation> — <https://github.com/flutter/flutter/blob/master/docs/src/content/ui/navigation/index.md>
- An introduction to widget testing — <https://docs.flutter.dev/cookbook/testing/widget/introduction> — <https://github.com/flutter/flutter/blob/master/docs/src/content/cookbook/testing/widget/introduction.md>
- Sách React Native trong repo (so sánh): Chương 2 — React Native cho Angular developer
- Package go_router: https://pub.dev/packages/go_router · package provider: <https://pub.dev/packages/provider>

Chương 3 — Dart cho TypeScript developer

Mục tiêu

- Đọc và viết Dart nhanh nhờ kiến thức TypeScript.
- Nắm các điểm Dart **khác** TypeScript: null safety chặt, class/constructor, `final/const`, record, pattern + `switch` expression, sealed class, extension, collection `if/for`.
- Biết tính năng mới nhất: **primary constructor** (Dart 3.13).
- Viết `async/await` với `Future`.

Giải thích đơn giản

Learning Pathway chính thức đặt bước "Dart Getting Started" trước Flutter, nhưng nói người đã quen ngôn ngữ hướng đối tượng hiện đại có thể đi nhanh. Bạn biết TypeScript, nên hãy nghĩ:

Dart $\approx$ TypeScript **có kiểu thật lúc chạy** + class kiểu Java/C# + pattern matching kiểu C#/Kotlin.

TypeScript	Dart	Ghi chú
<code>let x = 1 / const x = 1</code>	<code>var x = 1 / final x = 1</code>	Dart <code>const</code> = hằng lúc biên dịch
<code>string, number, boolean</code>	<code>String, int/double/num, bool</code>	<code>int</code> và <code>double</code> tách riêng
<code>string null</code>	<code>String?</code>	Null safety: mặc định không null
<code>x!</code>	<code>x!</code>	Giống nhau (ném lỗi nếu null)
<code>a ?? b, a?.b</code>	<code>a ?? b, a?.b</code>	Giống nhau
<code>`Hi \${name}`</code>	<code>'Hi \$name' / 'Hi \${user.name}'</code>	Chuỗi nháy đơn là quy ước
<code>interface</code>	<code>abstract interface class</code> / mọi class đều là interface ngầm	
<code>type A = B C (union)</code>	<code>sealed class</code> + các lớp con	Compiler kiểm tra đủ nhánh
<code>[a, b] tuple</code>	<code>Record (a, b) / (x: 1, y: 2)</code>	
<code>Promise<T></code>	<code>Future<T></code>	<code>async / await</code> y hệt
<code>Observable<T> (RxJS)</code>	<code>Stream<T></code>	Có sẵn trong ngôn ngữ (Tập 2)
<code>Array<T>, Map, Set</code>	<code>List<T>, Map<K, V>, Set<T></code>	
<code>arr.map(...).filter(...)</code>	<code>list.map(...).where(...).toList()</code>	<code>map</code> trả về Iterable lười (lazy)
<code>export/import</code>	<code>import 'package:app/x.dart';</code>	Tên bắt đầu <code>_</code> = private trong file
<code>npm</code>	<code>dart pub / flutter pub</code>	

Ví dụ

Toàn bộ ví dụ nằm trong một file Dart thuần: `examples/lib/chapters/ch03/dart_basics.dart`.

Null safety và record

```
/// `String?` có thể null; `??` giống TypeScript.
String greet(String? name) => 'Xin chào, ${name ?? 'bạn'}}!';

/// `int.tryParse` trả về `int?` (null nếu không phải số).
int? parseScore(String input) => int.tryParse(input.trim());

/// Trả về 2 giá trị một lúc mà không cần tạo class.
(int min, int max) minMax(List<int> numbers) {
  if (numbers.isEmpty) throw ArgumentError('Danh sách rỗng');
  var min = numbers.first, max = numbers.first;
  for (final n in numbers) {
    if (n < min) min = n;
    if (n > max) max = n;
  }
  return (min, max);
}

/// Record có tên: `({String word, String pos})`. Tách "negotiate (v)" → word + từ
loại.
({String word, String pos})? parseEntry(String raw) {
  final match = RegExp(r'^\s*(.+?)\s*\((\w+)\)\s*$').firstMatch(raw);
  if (match == null) return null;
  return (word: match.group(1)!, pos: match.group(2)!);
}
```

Dùng: `final (min, max) = minMax([5, 1, 9, 3]);` — đây là **destructuring** bằng pattern, giống `const [min, max] = ...` của TS. `r'...'` là chuỗi "raw" (không xử lý `\`), tiện cho RegExp.

Pattern + switch expression

```
String describeScore(int score) => switch (score) {
  < 0 || > 990 => 'Điểm không hợp lệ',
  >= 900 => 'Xuất sắc',
  >= 785 => 'Tốt',
  >= 600 => 'Khá',
  _ => 'Cần cố gắng',
};
```

`switch` expression **trả về giá trị**, không có `break`, và dùng được *relational pattern* (`>= 900`), *logical-or pattern* (`||`), và `_` (mặc định).

Sealed class — "discriminated union" có kiểm tra đủ nhánh

```
sealed class LoadState<T> {
    const LoadState();
}

final class Loading<T> extends LoadState<T> {
    const Loading();
}

final class Success<T> extends LoadState<T> {
    const Success(this.data);
    final T data;
}

final class Failure<T> extends LoadState<T> {
    const Failure(this.message);
    final String message;
}

/// Thiếu một nhánh → lỗi compile (exhaustiveness checking).
String renderState(LoadState<List<String>> state) => switch (state) {
    Loading() => 'Đang tải...',
    Success(data: final words) when words.isEmpty => 'Chưa có từ nào',
    Success(data: final words) => 'Có ${words.length} từ: ${words.join(', ')}',
    Failure(:final message) => 'Lỗi: $message',
};
```

Trong TS bạn viết `type LoadState = {kind: 'loading'} | {kind: 'success', data} | ...` rồi `switch (s.kind)`. Dart làm điều đó bằng `sealed class`: nếu bạn thêm lớp con mới mà quên xử lý, **compiler báo lỗi**. `Failure(:final message)` là cú pháp ngắn của `Failure(message: final message)`. Mẫu `Result` trong kiến trúc chính thức (Tập 3) dùng đúng kỹ thuật này.

Class và constructor

```
class Vocabulary {
    const Vocabulary(this.word, this.meaning, {this.level = 1});

    final String word;
    final String meaning;
    final int level;

    /// Named constructor – TS không có; thường dùng cho fromJson.
    Vocabulary.fromMap(Map<String, Object?> map)
        : word = map['word'] as String,
          meaning = map['meaning'] as String,
          level = (map['level'] as int?) ?? 1;
}
```

- `this.word` trong tham số = gán thẳng vào field (ngắn gọn).

- `{this.level = 1}` = **named parameter** tùy chọn có giá trị mặc định. Gọi: `Vocabulary('a', 'b', level: 2)`. Muốn bắt buộc: `{required this.level}`. Flutter dùng named parameter ở khắp nơi (`Text('x', style: ...)`).
- `const` constructor: tạo đối tượng hằng lúc biên dịch → Flutter tái sử dụng, không build lại (Tập 3).
- **Dart 3.13 — primary constructor** (giống "parameter properties" của TypeScript):

```
class const WordPair(final String english, final String vietnamese) {
  String get display => '$english → $vietnamese';
}
```

Một dòng khai báo cả field lẫn constructor. `class const` cho constructor `const`. Tính năng này cần `sdk: ^3.13.0` trong `pubspec.yaml` (dự án của sách: `^3.13.4`). Sách vẫn dùng cú pháp cổ điển là chính vì hầu hết tài liệu và package hiện nay viết theo kiểu đó.

Extension method — bỏ dấu tiếng Việt

```
extension VietnameseText on String {
  /// Bỏ dấu tiếng Việt: "Đàm phán" → "dam phan". Dùng cho tìm kiếm không dấu.
  String get withoutAccents {
    final buffer = StringBuffer();
    for (final ch in toLowerCase().split(' ')) {
      var out = ch;
      for (final entry in _accents.entries) {
        if (entry.value.contains(ch)) {
          out = entry.key;
          break;
        }
      }
      buffer.write(out);
    }
    return buffer.toString();
  }
}
```

(`_accents` là bảng `'a': 'àáâãä...'`, ... trong cùng file.) Gọi như một getter có sẵn: `'Đàm phán'.withoutAccents`. App TOEIC cần đúng hàm này để tìm từ "có dấu hoặc không dấu".

Collection if / for / spread

```
List<String> buildMenu({required bool loggedIn, List<String> extra = const []}) => [
  'Trang chủ',
  if (loggedIn) 'Hồ sơ' else 'Đăng nhập',
  for (final item in extra) item.toUpperCase(),
  ...['Giới thiệu'],
];
```

Đây là "template" của Flutter: `children: [ if (x) A(), for (...) B(), ]` thay cho `@if/@for`.

async / await

```
Future<Vocabulary> fetchWord(String word) async {  
  await Future<void>.delayed(const Duration(milliseconds: 10)); // giả lập gọi mạng  
  if (word.isEmpty) throw ArgumentError('word rỗng');  
  return Vocabulary(word, 'nghĩa của $word');  
}
```

Giống hệt TS. Khác: không có `Promise.all` mà là `Future.wait([...])`; lỗi bắt bằng `try/catch` hoặc `.catchError`. Tập 2, Chương 2 đi sâu về Future, Stream, isolate.

Kết quả test thật

`flutter test --reporter expanded test/chapters/ch03_test.dart` (2026-09-30):

```
00:00 +0: Chương 3 – Dart cho TypeScript developer null safety: ?? và tryParse  
00:00 +1: Chương 3 – Dart cho TypeScript developer record: minMax và parseEntry  
00:00 +2: Chương 3 – Dart cho TypeScript developer switch expression với relational pattern  
00:00 +3: Chương 3 – Dart cho TypeScript developer sealed class + pattern matching  
00:00 +4: Chương 3 – Dart cho TypeScript developer class: named constructor, primary  
constructor (Dart 3.13)  
00:00 +5: Chương 3 – Dart cho TypeScript developer extension: bỏ dấu tiếng Việt  
00:00 +6: Chương 3 – Dart cho TypeScript developer collection if / for / spread  
00:00 +7: Chương 3 – Dart cho TypeScript developer async/await  
00:00 +8: Chương 3 – Dart cho TypeScript developer Bài 1: diện tích Shape  
00:00 +9: Chương 3 – Dart cho TypeScript developer Bài 2: countWords  
00:00 +10: All tests passed!
```

Đi sâu

final VS const

- `final x = DateTime.now();` — gán **một lần**, giá trị có thể tính lúc chạy (giống `const` của TS).
- `const x = Duration(seconds: 1);` — giá trị phải biết **lúc biên dịch**; mọi `const` giống nhau là **cùng một đối tượng**. Trong Flutter, `const Text('Lưu')` giúp framework bỏ qua việc build lại.
- Lint `prefer_const_constructors` (trong `flutter_lints`) nhắc bạn thêm `const`.

Kiểu thật lúc chạy (reified generics)

TS xóa kiểu khi biên dịch sang JS; Dart **giữ** kiểu lúc chạy: `list is List<String>` hoạt động. Ép kiểu sai (`map['x'] as int` khi giá trị là chuỗi) ném `TypeError` ngay. Dự án của sách bật `strict-casts` và `strict-raw-types` trong `analysis_options.yaml` để bắt lỗi sớm hơn.

Class modifier

Dart 3 có `sealed`, `final`, `base`, `interface`, `abstract`, `mixin`. Người mới chỉ cần nhớ: `abstract interface class` $\approx$ `interface` của TS; `sealed` = union đóng; `final class` = không cho kế thừa bên ngoài thư viện.

Mọi thứ là object, không có `undefined`

Không có `undefined`, chỉ có `null`. Không có "truthy/falsy": `if (list)` là lỗi; phải viết `if (list.isNotEmpty)`. `==` gọi operator `==` (so sánh giá trị nếu class override, như `Task` ở Chương 9).

Lỗi và bẫy thường gặp

- **Dùng ! bừa bãi** để "tắt" lỗi `null` $\rightarrow$ crash lúc chạy. Ưu tiên `??`, `?.`, hoặc kiểm tra `if (x != null)`.
- **Quên `.toList()`** sau `map/where`: nhận `Iterable` lười; mỗi lần duyệt lại tính lại.
- **int chia int**: `7 / 2 == 3.5` (double); chia lấy nguyên dùng `7 ~/ 2 == 3`.
- **Truthy như JS**: `if (name)` không biên dịch.
- **Dart 3.13 breaking change**: không viết `final/var` trước tham số thường nữa (`void f(final int x)` lỗi) — hai từ này giờ dành cho primary constructor.
- **Chuỗi tiếng Việt dạng tổ hợp (NFD)**: "á" có thể là 1 ký tự (NFC) hoặc "a" + dấu rời (NFD). Hàm `withoutAccents` ở trên chỉ xử lý NFC — dữ liệu từ nguồn lạ nên chuẩn hóa trước.

Tóm tắt

- Dart $\approx$ TypeScript có kiểu thật lúc chạy, null safety chặt, class kiểu Java/C#.
- `final` (gán một lần) vs `const` (hằng biên dịch). Named parameter `{required ...}` có mặt khắp Flutter.
- Record + pattern + `switch expression` + `sealed class` = mô hình dữ liệu an toàn, gọn.
- Extension method thêm hàm vào kiểu có sẵn. Collection `if/for` là "template" của Flutter.
- Dart 3.13 có primary constructor.

Bài tập (có lời giải)

Bài 1. Viết `sealed class Shape` với `Circle(radius)` và `Rectangle(width, height)`, và hàm `area(Shape)` dùng `switch expression`.

▼ Lời giải

```
examples/lib/chapters/ch03/exercise_solution.dart:
```

```
sealed class Shape {
    const Shape();
}

final class Circle extends Shape {
    const Circle(this.radius);
    final double radius;
}

final class Rectangle extends Shape {
    const Rectangle(this.width, this.height);
    final double width, height;
}

double area(Shape shape) => switch (shape) {
    Circle(:final radius) => math.pi * radius * radius,
    Rectangle(:final width, :final height) => width * height,
};
```

Thử thêm `final class Triangle extends Shape {}` mà không sửa `area` → dart analyze báo lỗi (chạy thật):

```
error - bin/a.dart:4:25 - The type 'Shape' isn't exhaustively matched by the switch cases
since it doesn't match
the pattern 'Triangle()'. Try adding a wildcard pattern or cases that match 'Triangle()'. -
non_exhaustive_switch_expression
```

Đó là lợi ích của `sealed`.

Bài 2. Viết `countWords(String text) → Map<String, int>`: đếm số lần mỗi từ xuất hiện, không phân biệt hoa thường, bỏ dấu câu, giữ chữ tiếng Việt có dấu.

▼ Lời giải

```
Map<String, int> countWords(String text) {
    final counts = <String, int>{};
    final words = text.toLowerCase().split(RegExp(r"[^a-zA-ZÀ-ÿ']+")).where((w) =>
w.isNotEmpty);
    for (final w in words) {
        counts.update(w, (n) => n + 1, ifAbsent: () => 1);
    }
    return counts;
}
```

Test: `countWords('Deadline, deadline! Hạn chót.')` → `{'deadline': 2, 'hạn': 1, 'chót': 1}`.
`Map.update(..., ifAbsent:)` thay cho `counts[w] = (counts[w] ?? 0) + 1`. Dải ký tự `À-ÿ` bao gồm các chữ Latin có dấu dùng trong tiếng Việt (kể cả đ, U+0111).

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (dart.dev bị chặn trong sandbox; đọc file nguồn repo `dart-lang/site-www` ở commit `001b59a`):

- Records — <https://dart.dev/language/records> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/language/records.md>
- Patterns — <https://dart.dev/language/patterns> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/language/patterns.md>
- Branches (switch expression) — <https://dart.dev/language/branches> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/language/branches.md>
- Class modifiers — <https://dart.dev/language/class-modifiers> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/language/class-modifiers.md>
- Primary constructors — <https://dart.dev/language/primary-constructors> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/language/primary-constructors.md>
- Extension methods — <https://dart.dev/language/extension-methods> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/language/extension-methods.md>
- Collections — <https://dart.dev/language/collections> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/language/collections.md>
- Asynchronous programming — <https://dart.dev/language/async> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/language/async.md>
- Dart SDK CHANGELOG (3.13.0: primary constructors, breaking change `final/var`): <https://github.com/dart-lang/sdk/blob/main/CHANGELOG.md>
- Flutter for React Native developers (phần "Introduction to Dart for JavaScript Developers") — <https://docs.flutter.dev/flutter-for/react-native-devs> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/flutter-for/react-native-devs.md>
- Learning pathway (bước 2: Dart tutorial) — <https://docs.flutter.dev/learn/pathway> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/index.md>

Chương 4 — Widget: Stateless, Stateful, vòng đời và BuildContext

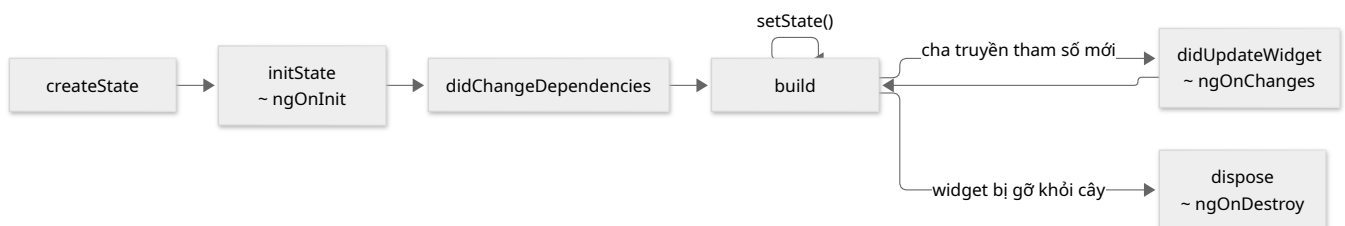
Mục tiêu

- Viết `StatelessWidget` và `StatefulWidget`; biết khi nào dùng loại nào.
- Hiểu vòng đời `State`: `initState` → `build` → `didUpdateWidget` → `dispose` (map sang hook của Angular).
- Hiểu **BuildContext** và **Key** ở mức đủ dùng.
- Dọn dẹp tài nguyên (Timer, controller) đúng cách, và test được điều đó.

Giải thích đơn giản

Docs chính thức ("Widget fundamentals" trong Learning Pathway) nói: widget là **mô tả** một phần giao diện, và widget **bất biến**. Muốn giao diện đổi thì Flutter tạo widget **mới** với dữ liệu mới.

- **StatelessWidget**: chỉ phụ thuộc tham số truyền vào. Giống component "dumb" chỉ có `@Input`.
- **StatefulWidget**: có một object `State` **sống lâu** đi kèm. Widget có thể bị tạo lại nhiều lần, nhưng `State` được giữ nguyên → dữ liệu trong `State` không mất. Gọi `setState` → Flutter gọi lại `build`.



Ví dụ

StatelessWidget — ProfileCard

`examples/lib/chapters/ch04/profile_card.dart`:

```

class ProfileCard extends StatelessWidget {
  const ProfileCard({
    super.key,
    required this.name,
    required this.role,
    required this.following,
    required this.onFollow,
  });

  final String name;
  final String role;
  final bool following;
  final VoidCallback onFollow;

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        leading: CircleAvatar(child: Text(name.characters.first)),
        title: Text(name),
        subtitle: Text(role),
        trailing: FilledButton.tonal(
          onPressed: onFollow,
          child: Text(following ? 'Đang theo dõi' : 'Theo dõi'),
        ),
      ),
    );
  }
}

```

ProfileCard không tự lưu following: cha (ProfileDemo, một StatefulWidget) giữ state và truyền xuống (**lifting state up**). name.characters.first lấy ký tự đầu an toàn với emoji/chữ có dấu (package characters đi kèm Flutter).

StatefulWidget có vòng đời — Clock

```
class Clock extends StatefulWidget {
  const Clock({super.key, this.paused = false, this.now = DateTime.now});

  final bool paused;

  /// Hàm lấy giờ hiện tại – truyền vào để test dễ (giống inject một ClockService).
  final DateTime Function() now;

  @override
  State<Clock> createState() => _ClockState();
}

class _ClockState extends State<Clock> {
  late DateTime _time = widget.now();
  Timer? _timer;

  @override
  void initState() {
    super.initState();
    if (!widget.paused) _start();
  }

  @override
  void didUpdateWidget(Clock oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.paused != widget.paused) {
      widget.paused ? _stop() : _start();
    }
  }

  void _start() {
    _timer?.cancel();
    _timer = Timer.periodic(const Duration(seconds: 1), (_) => setState(() => _time =
widget.now()));
  }

  void _stop() {
    _timer?.cancel();
    _timer = null;
  }

  @override
  void dispose() {
    _stop(); // quên dòng này → Timer vẫn chạy sau khi widget bị gỡ (rò rỉ bộ nhớ)
    super.dispose();
  }

  @override
  Widget build(BuildContext context) => Text(formatTime(_time), style:
Theme.of(context).textTheme.displaySmall);
}
```

- `widget.paused`: trong `State`, đọc tham số của widget qua `widget`. (giống `this.paused` của `@Input`).
- `didUpdateWidget` so sánh tham số cũ/mới — đúng vai trò `onChanges(changes)`.
- `late DateTime _time = widget.now();` — `late` cho phép khởi tạo lười, lúc lần đầu đọc (khi đó `widget` đã có).

Test vòng đời bằng thời gian giả

```
testWidgets('Clock cập nhật mỗi giây, dừng khi paused, hủy Timer khi dispose',
(tester) async {
  var now = DateTime(2026, 1, 1, 8);
  DateTime clock() => now;
  await pumpApp(tester, Clock(now: clock));
  expect(find.text('08:00:00'), findsOneWidget);
  now = now.add(const Duration(seconds: 2));
  await tester.pump(const Duration(seconds: 1));
  expect(find.text('08:00:02'), findsOneWidget);

  // Bài 2: paused = true → didUpdateWidget dừng Timer
  await pumpApp(tester, Clock(now: clock, paused: true));
  now = now.add(const Duration(seconds: 5));
  await tester.pump(const Duration(seconds: 3));
  expect(find.text('08:00:02'), findsOneWidget);
  // ... chạy lại, rồi gỡ widget:
  await tester.pumpWidget(const SizedBox());
  // Nếu Timer còn chạy sau dispose, flutter_test sẽ báo lỗi "A Timer is still
  pending".
});
```

Pump lại cùng loại widget ở cùng vị trí với tham số khác → Flutter **giữ State** và gọi `didUpdateWidget`. Đây là cách test "đổi `@Input`".

Kết quả thật (2026-09-30, `flutter test --reporter expanded test/chapters/ch04_test.dart`):

```
00:00 +0: Chương 4 – widget ProfileCard đổi nhãn khi bấm Theo dõi (state ở cha)
00:00 +1: Chương 4 – widget formatTime thêm số 0 phía trước
00:00 +2: Chương 4 – widget Clock cập nhật mỗi giây, dừng khi paused, hủy Timer khi dispose
00:01 +3: Chương 4 – widget Bài 1: LikeButton bật/tắt và đổi số
00:01 +4: All tests passed!
```

Xem trong app: tab **Lab** → "Ch.4".

Đi sâu

BuildContext là gì?

Mỗi widget trong cây có một `BuildContext` = **vị trí** của nó trong cây. Các hàm `X.of(context)` (ví dụ `Theme.of(context)`, `Navigator.of(context)`, `ScaffoldMessenger.of(context)`) **tìm lên trên** từ vị trí đó để lấy đối tượng gần nhất — giống injector phân cấp của Angular. Vì vậy:

- `Theme.of(context)` trong một widget nằm **dưới** `MaterialApp` mới tìm thấy theme.
- Sau `await`, `context` có thể không còn trong cây: kiểm tra `if (!context.mounted) return;` (với `State`: `if (!mounted) return;`) trước khi dùng. Chương 8 dùng mẫu này.

Widget, Element, RenderObject (đủ biết)

Flutter có 3 cây: **Widget** (mô tả, rẻ, tạo lại liên tục) → **Element** (giữ vị trí và State, sống lâu) → **RenderObject** (layout và vẽ). Bạn viết widget; Flutter tự so khớp widget mới với element cũ theo **loại (runtimeType) + key**. Cùng loại + cùng key → giữ element (và State), chỉ cập nhật.

Key — khi nào cần?

Hầu như không cần, **trừ** danh sách mà phần tử có thể đổi thứ tự/xóa (vuốt để xóa, sắp xếp). Khi đó dùng `ValueKey(item.id)` để Flutter giữ đúng State cho đúng phần tử — giống `track item.id` trong `@for` của Angular. Chương 6 và app mẫu dùng `ValueKey(task.id)` cho `Dismissible`.

Khi nào widget build lại?

1. `setState` trong State của nó.
2. Cha build lại và tạo widget mới cho nó (trừ khi widget là `const` — cùng đối tượng → bỏ qua).
3. Một `InheritedWidget` nó phụ thuộc (qua `X.of(context)`) thay đổi (Theme, MediaQuery...).

Mẹo: tách phần hay đổi ra widget nhỏ, và thêm `const` ở nơi có thể (Tập 3, Performance).

Lỗi và bẫy thường gặp

- Gọi `setState` trong `build` → vòng lặp vô hạn.
- Làm việc nặng hoặc gọi API trong `build` → chạy lại mỗi frame. Gọi trong `initState` hoặc trong `ViewModel` (Tập 2).
- Quên `dispose` `Timer` / `TextEditingController` / `AnimationController` / `StreamSubscription`.
- Dùng `context` sau `await` mà không kiểm tra `mounted`.
- Lưu dữ liệu vào field của `StatelessWidget` — widget bất biến; mọi field phải `final`.

- Quên gọi `super.initState()` / `super.dispose()` (lint `must_call_super` sẽ nhắc).

Tóm tắt

- Stateless = chỉ tham số; Stateful = có `State` sống lâu, đổi bằng `setState`.
- Vòng đời: `initState` (`~ngOnInit`), `didUpdateWidget` (`~ngOnChanges`), `dispose` (`~ngOnDestroy`).
- `BuildContext` = vị trí trong cây; `X.of(context)` tìm lên trên.
- Dùng `key` cho danh sách đối thứ tự. Luôn dọn dẹp tài nguyên trong `dispose`.

Bài tập (có lời giải)

Bài 1. Viết `LikeButton(initialLikes)`: bấm lần 1 → tim đầy và +1; bấm lần 2 → trở lại.

▼ Lời giải

`examples/lib/chapters/ch04/exercise_solution.dart`:

```
class _LikeButtonState extends State<LikeButton> {
  bool _liked = false;

  @override
  Widget build(BuildContext context) {
    final likes = widget.initialLikes + (_liked ? 1 : 0);
    return TextButton.icon(
      onPressed: () => setState(() => _liked = !_liked),
      icon: Icon(_liked ? Icons.favorite : Icons.favorite_border, semanticLabel:
        _liked ? 'Bỏ thích' : 'Thích'),
      label: Text('$likes'),
    );
  }
}
```

Chỉ **một** state `_liked`; số lượt thích là giá trị *suy ra* (derived) — tính trong `build`, giống `computed()`. Test: 12 → bấm → 13 (thấy icon `favorite`) → bấm → 12.

Bài 2. `Clock` chạy mãi. Thêm tham số `paused`; khi cha đổi `paused` thành `true` thì dừng `Timer`, `false` thì chạy lại.

▼ Lời giải

Dùng `didUpdateWidget` (đã có trong `Clock` ở trên):

```
@override
void didUpdateWidget(Clock oldWidget) {
  super.didUpdateWidget(oldWidget);
  if (oldWidget.paused != widget.paused) {
    widget.paused ? _stop() : _start();
  }
}
```

Và `initState` chỉ `_start()` khi `!widget.paused`. Test "Clock cập nhật mỗi giây, dừng khi paused..." pump lại `Clock(paused: true)`, cho 3 giây trôi qua và kiểm tra giờ **không** đổi; rồi `paused: false` và kiểm tra giờ chạy tiếp. Đây là mẫu "đăng ký lại khi input đổi" — giống `switchMap` của RxJS hoặc `useEffect(..., [paused])` của React.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Tutorial: Widget fundamentals — <https://docs.flutter.dev/learn/pathway/tutorial/widget-fundamentals> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/widget-fundamentals.md>
- Tutorial: Learn about stateful widgets — <https://docs.flutter.dev/learn/pathway/tutorial/stateful-widget> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/stateful-widget.md>
- Building user interfaces with Flutter — <https://docs.flutter.dev/ui> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/index.md>
- Adding interactivity — <https://docs.flutter.dev/ui/interactivity> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/interactivity/index.md>
- Flutter for React Native developers (StatelessWidget / StatefulWidget) — <https://docs.flutter.dev/flutter-for/react-native-devs> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/flutter-for/react-native-devs.md>

Chương 5 — Layout, styling, theming và adaptive layout

Mục tiêu

- Hiểu quy tắc layout của Flutter: **"Constraints go down. Sizes go up. Parent sets position."**
- Dùng `Row`, `Column`, `Expanded`, `Padding`, `SizedBox`, `Wrap`, `Card`, `ListTile`.
- Tạo theme Material 3 sáng/tối từ một màu gốc (`ColorScheme.fromSeed`) và bật **dark mode**.
- Làm layout thích ứng (adaptive) theo bề rộng màn hình bằng `LayoutBuilder`.
- Sửa lỗi kinh điển "RenderFlex overflowed".

Giải thích đơn giản

Không có CSS. Mỗi việc là một widget:

CSS / Angular	Flutter
<code>display:flex; flex-direction:row</code>	<code>Row</code>
<code>flex-direction:column</code>	<code>Column</code>
<code>justify-content</code> / <code>align-items</code>	<code>mainAxisAlignment</code> / <code>crossAxisAlignment</code>
<code>flex: 1</code>	<code>Expanded</code> (hoặc <code>Flexible</code>)
<code>padding</code>	<code>Padding(padding: EdgeInsets.all(16))</code>
<code>margin, gap</code>	<code>SizedBox(height: 8)</code> giữa các con, hoặc <code>spacing</code> của <code>Row/Column/Wrap</code>
<code>flex-wrap: wrap</code>	<code>Wrap</code>
<code>position:absolute</code>	<code>Stack</code> + <code>Positioned</code>
<code>@media (min-width: 600px)</code>	<code>LayoutBuilder</code> / <code>MediaQuery.sizeOf(context)</code>
CSS variables / theme SCSS	<code>ThemeData</code> , <code>ColorScheme</code> , <code>TextTheme</code>

Quy tắc vàng từ docs (trang "Understanding constraints"): **cha đưa ràng buộc (constraints) xuống cho con; con chọn kích thước trong ràng buộc đó và báo lên; cha quyết định vị trí của con.** Một widget không thể "muốn" to hơn chỗ cha cho — đó là nguồn gốc của lỗi overflow.

Ví dụ

Row / Column / Expanded — PriceCard

`examples/lib/chapters/ch05/layout_demo.dart` (rút gọn):

```
Card(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Row(
          children: [
            // Expanded chiếm hết chỗ còn lại (giống flex: 1).
            Expanded(child: Text(title, style: text.titleLarge)),
            Text(price, style: text.titleLarge?.copyWith(color: scheme.primary)),
          ],
        ),
        const SizedBox(height: 8),
        for (final f in features)
        Row(
          children: [
            Icon(Icons.check, size: 18, color: scheme.tertiary),
            const SizedBox(width: 6),
            Expanded(child: Text(f)),
          ],
        ),
      ],
    ),
  ),
);
```

`text` và `scheme` lấy từ `Theme.of(context).textTheme` / `.colorScheme` — không viết mã màu cứng.

Theme sáng/tối — examples/lib/theme.dart

```
abstract final class AppTheme {
  static const Color seed = Color(0xFF1D4ED8);

  static ThemeData light = _build(Brightness.light);
  static ThemeData dark = _build(Brightness.dark);

  static ThemeData _build(Brightness brightness) {
    final scheme = ColorScheme.fromSeed(seedColor: seed, brightness: brightness);
    return ThemeData(
      colorScheme: scheme,
      appBarTheme: AppBarTheme(backgroundColor: scheme.primaryContainer,
foregroundColor: scheme.onPrimaryContainer),
      inputDecorationTheme: const InputDecorationTheme(border: OutlineInputBorder()),
      listTileTheme: const ListTileThemeData(contentPadding:
EdgeInsets.symmetric(horizontal: 16)),
    );
  }
}
```

Và trong app.dart:

```
MaterialApp.router(
  theme: AppTheme.light,
  darkTheme: AppTheme.dark,
  themeMode: mode, // ThemeMode.system / light / dark – người dùng chọn ở tab
  Cài đặt
  routerConfig: _router,
)
```

- `ColorScheme.fromSeed` sinh cả bảng màu Material 3 (primary, secondary, tertiary, surface..., và các màu on... cho chữ) từ **một** màu gốc, cho cả sáng và tối.
- Material 3 là mặc định (`useMaterial3 == true` — test của sách kiểm tra điều này).
- `ThemeMode.system` theo cài đặt của iPhone (Settings → Display & Brightness).

Adaptive layout — số cột theo bề rộng

```
/// Adaptive layout: số cột theo bề rộng (breakpoint giống Material 3 window size classes).
int columnsForWidth(double width) {
  if (width >= 840) return 3; // expanded
  if (width >= 600) return 2; // medium
  return 1; // compact (điện thoại dọc)
}

class AdaptiveGrid extends StatelessWidget {
  const AdaptiveGrid({super.key, required this.children});

  final List<Widget> children;

  @override
  Widget build(BuildContext context) {
    // LayoutBuilder cho biết ràng buộc (constraints) mà cha trao cho widget này.
    return LayoutBuilder(
      builder: (context, constraints) {
        final columns = columnsForWidth(constraints.maxWidth);
        return GridView.count(
          crossAxisCount: columns,
          childAspectRatio: columns == 1 ? 2.4 : 1.4,
          padding: const EdgeInsets.all(8),
          children: children,
        );
      },
    );
  }
}
```

Docs ("Adaptive and responsive design") gợi ý ví dụ: dưới 600 px logic dùng bottom navigation bar, từ 600 trở lên dùng navigation rail. Mốc 840 lấy theo Material 3 window size classes (**UNVERIFIED** — trang m3.material.io không mở được trong sandbox).

Test đổi kích thước màn hình ảo:

```
testWidgets('AdaptiveGrid đổi số cột theo bề rộng', (tester) async {
  tester.view.physicalSize = const Size(1200, 800);
  tester.view.devicePixelRatio = 1;
  addTearDown(tester.view.reset);
  await pumpApp(tester, const LayoutDemo());
  final grid = tester.widget<GridView>(find.byType(GridView));
  final delegate = grid.gridDelegate as SliverGridDelegateWithFixedCrossAxisCount;
  expect(delegate.crossAxisCount, 3);
});
```

Kết quả thật (2026-09-30, flutter test --reporter expanded test/chapters/ch05_test.dart):

```
00:00 +0: Chương 5 – layout, theme, adaptive columnsForWidth theo breakpoint
00:00 +1: Chương 5 – layout, theme, adaptive AdaptiveGrid đổi số cột theo bề rộng
00:00 +2: Chương 5 – layout, theme, adaptive AppTheme: dark theme có Brightness.dark, cùng seed
00:01 +3: Chương 5 – layout, theme, adaptive Bài 1: colorForPriority lấy màu từ ColorScheme
00:01 +4: Chương 5 – layout, theme, adaptive Bài 2: chữ dài trong Row không gây overflow
00:01 +5: All tests passed!
```

Đi sâu

Vì sao `Text` dài trong `Row` bị tràn?

`Row` cho con **không giới hạn** chiều ngang (unbounded) để con tự đo. `Text` rất dài đo ra rộng hơn màn hình → `Row` không chứa nổi → vạch sọc vàng-đen và lỗi "A RenderFlex overflowed by N pixels". Cách sửa: bọc bằng `Expanded/Flexible` để con nhận ràng buộc **có giới hạn** (phần còn lại của `Row`), rồi `Text` tự xuống dòng hoặc cắt bằng `overflow: TextOverflow.ellipsis`.

MediaQuery VS LayoutBuilder

- `MediaQuery.sizeOf(context)` — kích thước **cả cửa sổ**.
- `LayoutBuilder` — kích thước **chỗ cha cho widget này** (chính xác hơn khi widget nằm trong một cột bên). Docs khuyên nghĩ theo không gian được cấp, không theo loại thiết bị.

Tùy biến theme theo component

`ThemeData` CÓ `appBarTheme`, `inputDecorationTheme`, `filledButtonTheme`, `cardTheme` ... Đặt ở một chỗ, cả app theo. Muốn một nhánh dùng theme khác: bọc bằng `Theme(data: ..., child: ...)` (giống `:host` trong Angular nhưng theo cây).

Material hay Cupertino?

Sách dùng Material 3 cho cả iOS (đơn giản, một bộ widget). Nhiều widget có bản `.adaptive` (ví dụ `Switch.adaptive`, `CircularProgressIndicator.adaptive`) tự dùng giao diện iOS trên iPhone.

Font tiếng Việt

Trên iPhone/Android, font hệ thống hiển thị tốt tiếng Việt. Trên **web**, Flutter tải font dự phòng từ Google Fonts khi gặp ký tự lạ; không có mạng thì có thể hiện ô vuông. Muốn chắc chắn, thêm font (ví dụ Noto Sans, giấy phép OFL) vào `assets` và khai báo trong `pubspec.yaml` (docs cookbook "Use a custom font").

Lỗi và bẫy thường gặp

- **Overflow:** Text dài trong Row → bọc Expanded.
- Expanded ngoài Row/Column/Flex → lỗi "Incorrect use of ParentDataWidget".
- ListView trong Column không có chiều cao → lỗi "Vertical viewport was given unbounded height". Bọc ListView bằng Expanded.
- **Viết màu cứng** (Colors.white) → dark mode xấu. Dùng colorScheme.onSurface, surface...
- **Quên const** cho widget tĩnh → build lại không cần thiết (lint nhắc).

Tóm tắt

- Constraints đi xuống, kích thước đi lên, cha đặt vị trí.
- Row/Column/Expanded ≈ Flexbox. Padding/SizedBox thay margin/padding.
- ColorScheme.fromSeed + theme/darkTheme/themeMode = dark mode đầy đủ.
- LayoutBuilder để thích ứng theo bề rộng.

Bài tập (có lời giải)

Bài 1. Viết colorForPriority(ColorScheme, Priority): cao → error, bình thường → primary, thấp → outline. Vì sao lấy từ ColorScheme thay vì Colors.red?

▼ Lời giải

examples/lib/chapters/ch05/exercise_solution.dart:

```
Color colorForPriority(ColorScheme scheme, Priority p) => switch (p) {  
  Priority.high => scheme.error,  
  Priority.normal => scheme.primary,  
  Priority.low => scheme.outline,  
};
```

Màu trong ColorScheme được tính cho cả sáng và tối, đủ tương phản với surface. Colors.red giữ nguyên ở dark mode nên có thể quá chói hoặc khó đọc. Test dùng AppTheme.dark.colorScheme để kiểm tra.

Bài 2. Dòng sau bị overflow trên iPhone nhỏ. Sửa để chữ bị cắt bằng "..." thay vì tràn:

```
Row(children: [Icon(Icons.label_outline), SizedBox(width: 8), Text(title),  
Icon(Icons.chevron_right)])
```

▼ Lời giải

```

Row(
  children: [
    const Icon(Icons.label_outline),
    const SizedBox(width: 8),
    Expanded(child: Text(title, maxLines: 1, overflow: TextOverflow.ellipsis)),
    const Icon(Icons.chevron_right),
  ],
)

```

Test đặt màn hình 320 px, `title` dài 240 ký tự, rồi `expect(tester.takeException(), isNull)` — nếu còn overflow, `takeException()` trả về lỗi `FlutterError`.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Tutorial: Layout widgets on a screen — <https://docs.flutter.dev/learn/pathway/tutorial/layout> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/layout.md>
- Layouts in Flutter — <https://docs.flutter.dev/ui/layout> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/layout/index.md>
- Understanding constraints — <https://docs.flutter.dev/ui/layout/constraints> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/layout/constraints.md>
- Organize a theme (cookbook) — <https://docs.flutter.dev/cookbook/design/themes> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/design/themes.md>
- Tutorial: Adaptive layouts — <https://docs.flutter.dev/learn/pathway/tutorial/adaptive-layout> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/adaptive-layout.md>
- Adaptive and responsive design — general approach — <https://docs.flutter.dev/ui/adaptive-responsive/general> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/adaptive-responsive/general.md>
- Use a custom font (cookbook) — <https://docs.flutter.dev/cookbook/design/fonts> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/design/fonts.md>
- Flutter for web developers — <https://docs.flutter.dev/flutter-for/web-devs> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/flutter-for/web-devs.md>

Chương 6 — Danh sách và cuộn: ListView, GridView, Dismissible, slivers

Mục tiêu

- Hiển thị danh sách dài hiệu quả bằng `ListView.builder` (chỉ tạo dòng đang hiện).
- Dùng `ListView.separated`, `GridView`, `Dismissible` (vuốt để xóa).
- Làm màn hình cuộn tùy biến bằng `CustomScrollView` + **slivers** (app bar co giãn, danh sách có tiêu đề nhóm).
- Test cuộn và vuốt.

Giải thích đơn giản

Angular / RN	Flutter
@for với vài phần tử	Column(children: [for (...) ...]) (trong SingleChildScrollView nếu cần cuộn)
CDK virtual scroll / RN FlatList	ListView.builder
RN SectionList	CustomScrollView + nhiều SliverList + tiêu đề SliverToBoxAdapter
CSS grid	GridView.count / GridView.builder
trackBy / key	ValueKey(item.id)

`ListView.builder` nhận `itemCount` và hàm `itemBuilder(context, index)`. Flutter chỉ gọi hàm cho các dòng **đang nằm trong màn hình** (và một ít vùng đệm) — nên 1.000 hay 100.000 dòng vẫn mượt. Docs cookbook "Work with long lists" dùng đúng cách này.

Ví dụ

Danh sách dài — ContactList

`examples/lib/chapters/ch06/lists_demo.dart`:

```

class ContactList extends StatelessWidget {
  const ContactList({super.key, this.count = 1000});

  final int count;

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      itemCount: count,
      itemExtent: 56, // biết trước chiều cao → cuộn nhanh hơn
      itemBuilder: (context, i) => ListTile(
        leading: CircleAvatar(child: Text('${i % 26} + 1')),
        title: Text('Liên hệ ${i + 1}'),
      ),
    );
  }
}

```

Test chứng minh tính "lười" (lazy):

```

testWidgets('ContactList chỉ build dòng đang hiện (lazy) và cuộn tới dòng 200',
  (tester) async {
    await pumpApp(tester, const ContactList());
    expect(find.text('Liên hệ 1'), findsOneWidget);
    expect(find.text('Liên hệ 200'), findsNothing); // chưa được tạo
    await tester.scrollUntilVisible(find.text('Liên hệ 200'), 500);
    expect(find.text('Liên hệ 200'), findsOneWidget);
  });

```

Vuốt để xóa — Dismissible

```

return ListView.separated(
  itemCount: _items.length,
  separatorBuilder: (context, i) => const Divider(height: 1),
  itemBuilder: (context, i) {
    final item = _items[i];
    return Dismissible(
      key: ValueKey(item),
      onDismissed: (_) => setState(() => _items.remove(item)),
      background: ColoredBox(color: Theme.of(context).colorScheme.errorContainer),
      child: ListTile(title: Text(item)),
    );
  },
);

```

`Dismissible` **bắt buộc** có `key` duy nhất, ổn định. Nếu dùng `ValueKey(i)` (theo index), sau khi xóa dòng 2 thì dòng 3 nhận key "2" → Flutter nhầm, báo lỗi "A dismissed Dismissible widget is still part of the tree". App mẫu (Chương 9) thêm `SnackBar` "Hoàn tác".

Slivers — danh sách từ vựng có tiêu đề nhóm

```
return CustomScrollView(
  slivers: [
    const SliverAppBar.medium(title: Text('Từ vựng theo chữ cái'), pinned: true,
      automaticallyImplyLeading: false),
    for (final entry in groups.entries) ...[
      SliverToBoxAdapter(
        child: Padding(
          padding: const EdgeInsets.fromLTRB(16, 16, 16, 4),
          child: Text(entry.key, style: text.titleMedium),
        ),
      ),
      SliverList.builder(
        itemCount: entry.value.length,
        itemBuilder: (context, i) => ListTile(title: Text(entry.value[i])),
      ),
    ],
  ],
);
```

Learning Pathway có bài "Scrolling and slivers": sliver là **mảnh** của một vùng cuộn. Ghép nhiều sliver (app bar co giãn, lưới, danh sách) vào **một** `CustomScrollView` để cuộn chung. App TOEIC có thể dùng mẫu này cho "từ vựng theo unit".

Kết quả thật (2026-09-30, `flutter test --reporter expanded test/chapters/ch06_test.dart`):

```
00:00 +0: Chương 6 – danh sách ContactList chỉ build dòng đang hiện (lazy) và cuộn tới dòng 200
00:01 +1: Chương 6 – danh sách SwipeList: vuốt để xóa
00:02 +2: Chương 6 – danh sách SectionedWordList hiển thị tiêu đề nhóm
00:02 +3: Chương 6 – danh sách Bài 1: groupByInitial sắp xếp nhóm và từ
00:02 +4: Chương 6 – danh sách Bài 2: ColorGrid có 20 ô
00:02 +5: All tests passed!
```

Đi sâu

ListView có mấy kiểu?

Constructor	Dùng khi
<code>ListView(children: [...])</code>	Ít phần tử, biết trước (tạo hết ngay)
<code>ListView.builder</code>	Nhiều / không biết trước — lazy
<code>ListView.separated</code>	Lazy + có đường kẻ giữa các dòng
<code>ListView.custom</code>	Tùy biến sâu (hiếm dùng)

`itemExtent` (hoặc `prototypeItem`) giúp Flutter không phải đo từng dòng → nhảy tới vị trí xa nhanh hơn.

Cuộn và bàn phím, SafeArea

`Scaffold` tự tránh bàn phím (`resizeToAvoidBottomInset`) và `AppBar/NavigationBar` tự tránh tai thỏ. Nếu tự dựng màn hình không có `Scaffold`, bọc bằng `SafeArea`.

Kéo để làm mới, cuộn vô hạn

- `RefreshIndicator(onRefresh: () async {...}, child: ListView(...))` — kéo xuống để tải lại.
- Cuộn vô hạn: trong `itemBuilder`, khi `index` gần cuối thì gọi tải trang tiếp (Tập 2, HTTP).

Hiệu năng

Danh sách lớn chậm thường do: tạo widget nặng trong mỗi dòng, ảnh lớn không cache, hoặc `shrinkWrap: true` (bắt `ListView` đo **mọi** phần tử — mất tính lazy). Tập 3 (Performance) có thêm.

Lỗi và bẫy thường gặp

- `ListView` trong `Column` không có `Expanded` → "unbounded height".
- `shrinkWrap: true` cho danh sách dài → mất lazy, giật.
- **Key theo index** cho `Dismissible` / danh sách sắp xếp lại → State bị lẫn.
- **Sửa list trong `onDismissed` mà quên `setState`** → lỗi "dismissed Dismissible still in tree".
- **Lồng `ListView` cùng chiều** mà không dùng `sliver` → cuộn "giành nhau".

Tóm tắt

- `ListView.builder` = danh sách lazy; `separated` có đường kẻ; `GridView` cho lưới.
- `Dismissible` + `ValueKey(id)` để vuốt xóa.
- `CustomScrollView` + `slivers` cho màn hình cuộn phức tạp.
- Test cuộn bằng `scrollUntilVisible`, vuốt bằng `tester.drag`.

Bài tập (có lời giải)

Bài 1. Viết `groupByInitial(words)` → `Map<String, List<String>>`: nhóm theo chữ cái đầu (viết hoa), bỏ chuỗi rỗng, các nhóm A → Z, từ trong nhóm A → Z không phân biệt hoa thường. Dùng kết quả cho `SectionedWordList`.

▼ Lời giải

```
examples/lib/chapters/ch06/exercise_solution.dart:
```

```

Map<String, List<String>> groupByInitial(Iterable<String> words) {
  final groups = <String, List<String>>{};
  for (final w in words.map((w) => w.trim()).where((w) => w.isNotEmpty)) {
    final key = w.characters.first.toUpperCase();
    groups.putIfAbsent(key, () => []).add(w);
  }
  final keys = groups.keys.toList()..sort();
  return {for (final k in keys) k: (groups[k]!..sort((a, b) =>
a.toLowerCase().compareTo(b.toLowerCase())))};
}

```

`..sort()` là **cascade**: gọi `sort()` trên list rồi trả về chính list đó. Map literal của Dart **giữ thứ tự chèn**, nên thứ tự khóa là A→Z. Test: `['delay', 'Budget', 'approve', 'agenda', 'deadline']` → `{A: [agenda, approve], B: [Budget], D: [deadline, delay]}`.

Bài 2. Làm lưới 20 ô màu; số cột = `columnsForWidth(bề rộng) + 1` (dùng lại hàm của Chương 5).

▼ Lời giải

```

LayoutBuilder(
  builder: (context, constraints) => GridView.builder(
    padding: const EdgeInsets.all(8),
    gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
      crossAxisCount: columnsForWidth(constraints.maxWidth) + 1,
      mainAxisSpacing: 8,
      crossAxisSpacing: 8,
    ),
    itemCount: 20,
    itemBuilder: (context, i) => Container(
      color: Colors.primaryes[i % Colors.primaryes.length],
      alignment: Alignment.center,
      child: Text('Ô ${i + 1}', style: const TextStyle(color: Colors.white)),
    ),
  ),
)

```

`GridView.builder` cũng lazy như `ListView.builder`. Test kiểm tra thấy "Ô 1" (`ch06_test.dart`).

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Cookbook: Use lists — <https://docs.flutter.dev/cookbook/lists/basic-list> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/lists/basic-list.md>

- Cookbook: Work with long lists — <https://docs.flutter.dev/cookbook/lists/long-lists> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/lists/long-lists.md>
- Cookbook: Create a grid list — <https://docs.flutter.dev/cookbook/lists/grid-lists> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/lists/grid-lists.md>
- Cookbook: Implement swipe to dismiss — <https://docs.flutter.dev/cookbook/gestures/dismissible> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/gestures/dismissible.md>
- Tutorial: Scrolling and slivers — <https://docs.flutter.dev/learn/pathway/tutorial/slivers> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/slivers.md>
- Flutter for React Native developers (FlatList/SectionList) — <https://docs.flutter.dev/flutter-for/react-native-devs> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/flutter-for/react-native-devs.md>

Chương 7 — Form và nhập liệu

Mục tiêu

- Dùng `TextField`, `TextFormField`, `TextEditingController`, `FocusNode` / `textInputAction`.
- Xây form có kiểm tra (validation) bằng `Form` + `GlobalKey<FormState>` + `validator`.
- Viết validator là **hàm thuần** để test dễ (giống `Validators` của Angular).
- Hiện/ẩn mật khẩu, bàn phím email, nút "Tiếp" trên bàn phím iPhone.

Giải thích đơn giản

Angular Reactive Forms	Flutter
<code>FormGroup</code>	<code>Form</code> + <code>GlobalKey<FormState></code>
<code>FormControl</code> + <code><input formControlName></code>	<code>TextFormField</code> (+ <code>TextEditingController</code> để đọc/ghi giá trị)
<code>Validators.required</code>	<code>validator: (value) => value!.isEmpty ? 'Bắt buộc' : null</code>
<code>Validators.compose([...])</code>	hàm <code>compose</code> tự viết (bên dưới)
<code>form.markAllAsTouched(); if (form.valid)</code>	<code>if (formKey.currentState!.validate())</code>
<code>(input) / valueChanges</code>	<code>onChanged: / controller.addListener</code>

Quy ước của Flutter: validator trả về **chuỗi lỗi** nếu sai, `null` nếu đúng. `Form` gom mọi `TextFormField` bên dưới; `validate()` chạy hết validator và hiện lỗi ngay dưới từng ô.

Ví dụ

Validator là hàm thuần — examples/lib/chapters/ch07/validators.dart

```
typedef Validator = String? Function(String? value);

Validator requiredField([String message = 'Bắt buộc nhập']) =>
  (value) => (value == null || value.trim().isEmpty) ? message : null;

Validator minLength(int n) => (value) => (value ?? '').trim().length < n ? 'Tối thiểu
$n ký tự' : null;

final _emailRe = RegExp(r'^[@\s]+@[@\s]+\.[@ \s]+$');

Validator email() =>
  (value) => (value == null || value.isEmpty || !_emailRe.hasMatch(value.trim())) ?
  null : 'Email không hợp lệ';

/// Ghép nhiều validator: trả về lỗi đầu tiên (giống `Validators.compose`).
Validator compose(List<Validator> validators) => (value) {
  for (final v in validators) {
    final error = v(value);
    if (error != null) return error;
  }
  return null;
};
```

`typedef` đặt tên cho một kiểu hàm. `[String message = ...]` là **tham số vị trí tùy chọn** (khác `{}` là tham số có tên).

Form đăng ký — sign_up_form.dart (rút gọn)

```
class _SignUpFormState extends State<SignUpForm> {
  final _formKey = GlobalKey<FormState>();
  final _name = TextEditingController();
  final _email = TextEditingController();
  final _password = TextEditingController();
  bool _obscure = true;

  @override
  void dispose() {
    _name.dispose();
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

  void _submit() {
    // validate() chạy mọi validator và hiện lỗi dưới từng ô (giống
    markAllAsTouched()).
    if (_formKey.currentState!.validate()) {
      widget.onSubmit(SignUpData(name: _name.text.trim(), email: _email.text.trim()));
    }
  }

  @override
  Widget build(BuildContext context) {
    return Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          TextFormField(
            controller: _name,
            decoration: const InputDecoration(labelText: 'Họ tên'),
            textInputAction: TextInputAction.next, // phím "Tiếp" trên bàn phím iPhone
            validator: compose([requiredField(), minLength(2)]),
          ),
          TextFormField(
            controller: _email,
            decoration: const InputDecoration(labelText: 'Email'),
            keyboardType: TextInputType.emailAddress,
            textInputAction: TextInputAction.next,
            autocorrect: false,
            validator: compose([requiredField(), email()]),
          ),
          TextFormField(
            controller: _password,
            decoration: InputDecoration(
              labelText: 'Mật khẩu',
              suffixIcon: IconButton(
                tooltip: _obscure ? 'Hiện mật khẩu' : 'Ẩn mật khẩu',
                icon: Icon(_obscure ? Icons.visibility : Icons.visibility_off),
                onPressed: () => setState(() => _obscure = !_obscure),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

```

        obscureText: _obscure,
        textInputAction: TextInputAction.done,
        onFieldSubmitted: (_) => _submit(),
        validator: compose([requiredField(), minLength(8)]),
      ),
      FilledButton(onPressed: _submit, child: const Text('Đăng ký')),
    ],
  ),
);
}
}

```

(Bản đầy đủ có thêm các `SizeBox` khoảng cách.) `TextInputAction.next` làm bàn phím hiện nút "Tiếp" và tự chuyển focus sang ô kế tiếp; `.done` hiện "Xong" và gọi `onFieldSubmitted`.

Test form

```

testWidgets('SignUpForm hiện lỗi, rồi gửi dữ liệu khi hợp lệ', (tester) async {
  SignUpData? sent;
  await pumpApp(tester, SignUpForm(onSubmit: (d) => sent = d));
  await tester.tap(find.text('Đăng ký'));
  await tester.pump();
  expect(find.text('Bắt buộc nhập'), findsNWidgets(3));
  expect(sent, isNull);

  await tester.enterText(find.widgetWithText(TextFormField, 'Họ tên'), 'Nobin');
  await tester.enterText(find.widgetWithText(TextFormField, 'Email'),
'nobin@example.com');
  await tester.enterText(find.widgetWithText(TextFormField, 'Mật khẩu'),
'matkhau123');
  await tester.tap(find.text('Đăng ký'));
  await tester.pump();
  expect(sent?.name, 'Nobin');
});

```

Kết quả thật (2026-09-30, `flutter test --reporter expanded test/chapters/ch07_test.dart`):

```

00:00 +0: Chương 7 – form validators thuần: requiredField, email, minLength, compose
00:00 +1: Chương 7 – form SignUpForm hiện lỗi, rồi gửi dữ liệu khi hợp lệ
00:01 +2: Chương 7 – form nút mắt bật/tắt ẩn mật khẩu
00:01 +3: Chương 7 – form Bài 1: matchesField so sánh với giá trị hiện tại của ô khác
00:01 +4: Chương 7 – form Bài 2: ChangePasswordForm chỉ gọi onDone khi hai ô khớp
00:01 +5: All tests passed!

```

Đi sâu

TextField hay TextFormField?

- `TextField`: ô nhập đơn lẻ (tìm kiếm, chat). Đọc giá trị qua `onChanged` hoặc controller.
- `TextFormField`: `TextField` + tích hợp `Form` (validator, `onSaved`, `reset`). Dùng trong form.

Khi nào hiện lỗi?

`autovalidateMode`:

- `disabled` (mặc định): chỉ khi gọi `validate()`.
- `onUserInteraction`: sau khi người dùng gõ vào ô đó (giống `touched && invalid`). App mẫu dùng mode này.
- `always`: luôn luôn (dễ làm người dùng khó chịu ngay khi mở màn hình).

Controller và vòng đời

`TextEditingController` giữ văn bản + vị trí con trỏ và **phải** `dispose()`. Tạo nó trong `State` (không tạo trong `build`, nếu không mỗi lần `build` sẽ mất chữ đang gõ). Đặt giá trị ban đầu: `TextEditingController(text: task.title)` — app mẫu làm vậy ở màn hình "Sửa việc".

Validator phụ thuộc ô khác

Validator nhận **hàm đọc** giá trị ô kia `() => _password.text`), không nhận chuỗi cố định — vì validator được tạo một lần khi `build`, còn giá trị thì thay đổi (Bài 1).

Thư viện form

Docs cookbook dùng `Form` có sẵn. Form lớn có thể dùng package bên ngoài (ví dụ `flutter_form_builder`, `reactive_forms` — package sau mô phỏng Reactive Forms của Angular). Sách **không** dùng để giữ ít phụ thuộc; chưa kiểm chứng phiên bản.

Lỗi và bẫy thường gặp

- **Tạo controller trong `build`** → mất chữ khi rebuild, rò rỉ bộ nhớ.
- **Quên `dispose` controller.**
- **Validator trả về `''` (chuỗi rỗng)** thay vì `null` → Flutter coi là lỗi (hiện dòng trống đỏ).
- **Tên hàm trùng thư viện test:** sách đặt tên validator là `matches` và gặp lỗi thật khi chạy test: `'matches' is imported from both 'package:matcher/src/string_matchers.dart' and '.../exercise_solution.dart'` → đổi tên thành `matchesField` (hoặc dùng `import ... as`).
- **Bàn phím che ô nhập** khi không dùng `Scaffold/ListView` → bọc form trong `ListView` hoặc `SingleChildScrollView`.

Tóm tắt

- `Form` + `GlobalKey<FormState>` + `TextFormField(validator:)` ≈ Reactive Forms.
- Validator = hàm thuần `String? Function(String?)`; ghép bằng `compose`; test không cần UI.

- Controller phải `dispose`. `textInputAction` + `keyboardType` cho trải nghiệm bàn phím tốt trên iPhone.

Bài tập (có lời giải)

Bài 1. Viết validator `matchesField(other)` cho ô "Nhập lại mật khẩu": lỗi "Mật khẩu không khớp" khi khác giá trị ô mật khẩu.

▼ Lời giải

`examples/lib/chapters/ch07/exercise_solution.dart`:

```
Validator matchesField(String Function() other, [String message = 'Mật khẩu không khớp']) =>
    (value) => value == other() ? null : message;
```

Test: `password = 'abc12345'` → hợp lệ; đổi `password = 'khac'` → cùng validator giờ báo lỗi. Điều này chứng minh validator đọc giá trị **hiện tại** chứ không chụp giá trị cũ.

Bài 2. Làm form "Đổi mật khẩu" gồm hai ô dùng `matchesField`; chỉ gọi `onDone` khi hợp lệ.

▼ Lời giải

```
Form(
  key: _formKey,
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      TextFormField(
        controller: _password,
        obscureText: true,
        decoration: const InputDecoration(labelText: 'Mật khẩu mới'),
        validator: compose([requiredField(), minLength(8)]),
      ),
      TextFormField(
        obscureText: true,
        decoration: const InputDecoration(labelText: 'Nhập lại mật khẩu'),
        validator: matchesField(() => _password.text),
      ),
      FilledButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) widget.onDone();
        },
        child: const Text('Đổi mật khẩu'),
      ),
    ],
  ),
)
```

Test nhập `matkhau123` / `matkhau999` → thấy "Mật khẩu không khớp", `onDone` chưa gọi; sửa ô 2 thành `matkhau123` → `onDone` được gọi.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Tutorial: Handle user input — <https://docs.flutter.dev/learn/pathway/tutorial/user-input> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/user-input.md>
- Cookbook: Build a form with validation — <https://docs.flutter.dev/cookbook/forms/validation> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/forms/validation.md>
- Cookbook: Retrieve the value of a text field — <https://docs.flutter.dev/cookbook/forms/retrieve-input> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/forms/retrieve-input.md>
- Cookbook: Focus and text fields — <https://docs.flutter.dev/cookbook/forms/focus> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/forms/focus.md>
- Cookbook: Handle changes to a text field — <https://docs.flutter.dev/cookbook/forms/text-field-changes> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/forms/text-field-changes.md>
- Flutter for React Native developers (Form input) — <https://docs.flutter.dev/flutter-for/react-native-devs> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/flutter-for/react-native-devs.md>

Chương 8 — Điều hướng: Navigator và go_router

Mục tiêu

- Dùng `Navigator.push/pop` (mệnh lệnh) và nhận kết quả trả về từ màn hình khác.
- Dùng **go_router** (khai báo, theo URL) — cách docs chính thức khuyên cho đa số app: path parameter, query parameter, route lồng, tab giữ state (`StatefulShellRoute`), trang 404.
- Viết **guard** bằng `redirect + refreshListenable`.
- Biết khác nhau giữa `context.go` và `context.push`.

Giải thích đơn giản

Flutter có 2 kiểu điều hướng (docs "Navigation and routing"):

1. **Navigator (imperative)**: một ngăn xếp (stack) màn hình. `push` chồng màn hình mới lên, `pop` bỏ ra. Đơn giản, nhưng không có URL → khó deep link, không hợp web.
2. **Router (declarative)** qua package như **go_router**: bạn khai báo bằng URL → **màn hình**; đi đến URL nào thì stack được dựng theo URL đó. Docs: `go_router` là "preferred way to write 90% of Flutter applications"; named routes (`Navigator.pushNamed`) **không** được khuyên dùng cho đa số app.

Angular Router	go_router
Routes array	<code>GoRouter(routes: [GoRoute(...)])</code>
<code>path: 'tasks/:id'</code>	<code>GoRoute(path: ':id')</code> lồng trong <code>/tasks</code>
<code>route.paramMap.get('id')</code>	<code>state.pathParameters['id']</code>
<code>route.queryParamMap</code>	<code>state.uri.queryParameters</code>
<code>router.navigateByUrl('/tasks/1')</code>	<code>context.go('/tasks/1')</code>
<code>children + <router-outlet></code>	route con / <code>ShellRoute</code> / <code>StatefulShellRoute</code>
<code>canActivate</code> trả <code>UrlTree</code>	<code>redirect</code> : trả URL mới hoặc <code>null</code>
<code>**</code> → <code>NotFound</code>	<code>errorBuilder</code> :

Ví dụ

Navigator: push và nhận kết quả

`examples/lib/chapters/ch08/navigator_basics.dart`:

```

Future<void> _pick() async {
  // push trả về Future: hoàn thành khi màn hình kia gọi Navigator.pop(context,
  value).

  final result = await Navigator.of(context).push<String>(
    MaterialPageRoute(builder: (context) => const ColorChoiceScreen()),
  );
  if (!mounted) return; // widget có thể đã bị gỡ trong lúc chờ
  setState(() => _picked = result ?? 'đã hủy');
}

```

Màn hình kia: `ListFile(title: Text(c), onTap: () => Navigator.pop(context, c))`. Người dùng bấm nút Back → `result == null` ("đã hủy"). Hộp thoại `showDialog` cũng trả kết quả theo cách này (app mẫu dùng để xác nhận xóa).

go_router: path, query, guard

`examples/lib/chapters/ch08/word_router.dart`:

```

GoRouter buildWordRouter({required ValueNotifier<bool> loggedIn, String
initialLocation = '/'}) {
  return GoRouter(
    initialLocation: initialLocation,
    refreshListenable: loggedIn, // loggedIn đổi → chạy lại redirect
    redirect: (context, state) {
      final goingToWords = state.matchedLocation.startsWith('/words');
      if (goingToWords && !loggedIn.value) {
        // giống CanActivateFn trả về UriTree('/login')
        return Uri(path: '/login', queryParameters: {'from':
state.uri.toString()}).toString();
      }
      return null; // null = cho đi tiếp
    },
    routes: [
      GoRoute(path: '/', builder: (context, state) => const WordHome()),
      GoRoute(
        path: '/words/:id',
        builder: (context, state) =>
          WordDetail(id: state.pathParameters['id']!, tab:
state.uri.queryParameters['tab'] ?? 'meaning'),
      ),
      GoRoute(
        path: '/login',
        builder: (context, state) => LoginScreen(loggedIn: loggedIn, from:
state.uri.queryParameters['from']),
      ),
    ],
  );
}

```

Dùng trong app: `MaterialApp.router(routerConfig: router)`. Mở từ: `context.push('/words/1')` hoặc `context.push('/words/1?tab=examples')`. Sau khi đăng nhập: `loggedIn.value = true;`
`context.go(from ?? '/')`.

App mẫu: 3 tab giữ state + route lồng

`examples/lib/router.dart` (rút gọn):

```
GoRouter(  
  initialLocation: initialLocation,  
  errorBuilder: (context, state) => NotFoundScreen(location: state.uri.toString()),  
  routes: [  
    StatefulShellRoute.indexedStack(  
      builder: (context, state, shell) => HomeShell(shell: shell),  
      branches: [  
        StatefulShellBranch(routes: [  
          GoRoute(  
            path: '/tasks',  
            builder: (context, state) => TaskListScreen(store: store),  
            routes: [  
              GoRoute(path: 'new', builder: (context, state) => TaskFormScreen(store:  
store)),  
              GoRoute(  
                path: ':id',  
                builder: (context, state) => TaskDetailScreen(store: store, id:  
state.pathParameters['id']!),  
                routes: [  
                  GoRoute(path: 'edit', builder: (context, state) =>  
TaskFormScreen(store: store, editId:  
state.pathParameters['id'])),  
                ],  
              ),  
            ],  
          ),  
        ],  
      ),  
    ],  
    StatefulShellBranch(routes: [/* /lab, /lab/:chapter */]),  
    StatefulShellBranch(routes: [/* /settings */]),  
  ],  
)
```

`HomeShell` vẽ `NavigationBar` và gọi `shell.goBranch(i)` khi đổi tab. `indexedStack` giữ **mỗi tab một stack** riêng: sang tab Cài đặt rồi quay lại, danh sách vẫn ở nguyên vị trí (test "Cài đặt: chuyển sang giao diện Tối" kiểm tra).

Kết quả thật (2026-09-30, `flutter test --reporter expanded test/chapters/ch08_test.dart`):

```
00:00 +0: Chương 8 – điều hướng Navigator.push nhận kết quả khi màn hình kia pop
00:01 +1: Chương 8 – điều hướng go_router: path + query parameter
00:01 +2: Chương 8 – điều hướng go_router: redirect về /login khi chưa đăng nhập, rồi quay lại
00:01 +3: Chương 8 – điều hướng Bài 1: safeRedirectTarget chặn open redirect
00:01 +4: All tests passed!
```

Đi sâu

go hay push?

- `context.go('/tasks/t2')`: **đặt** vị trí theo URL. `go_router` dựng stack từ cây route: `/tasks` rồi `/tasks/t2` → có nút Back về danh sách. URL trên web đổi theo.
- `context.push('/words/1')`: **chồng** route lên stack hiện tại, và trả về `Future` (có thể nhận kết quả như `Navigator.push`). Hợp với màn hình tạm (chi tiết, chọn một giá trị).
- `context.pop()` bỏ màn hình trên cùng; kiểm tra `context.canPop()` trước nếu màn hình có thể được mở bằng deep link (app mẫu: nút "Lưu" dùng `canPop() ? pop() : go('/tasks')`).

Page-backed và pageless

Docs giải thích: route tạo bởi `Router/go_router` là *page-backed* (deep link được); route tạo bằng `Navigator.push/showDialog` là *pageless*. Khi một route *page-backed* bị bỏ khỏi stack (vì `go` sang URL khác), các route *pageless* phía sau nó cũng bị bỏ. Vì vậy: trong app dùng `go_router`, dùng `Navigator.push` chỉ cho thứ tạm thời (hộp thoại, bottom sheet).

Thứ tự route và "new" vs ":id"

`go_router` so khớp theo thứ tự khai báo. Đặt `GoRoute(path: 'new')` **trước** `GoRoute(path: ':id')`, nếu không `/tasks/new` sẽ khớp `:id = "new"`.

Deep link trên iPhone

Với `go_router`, mở app từ URL (universal link / custom scheme) sẽ dựng đúng màn hình. Cấu hình iOS (Associated Domains) và Android (intent filter) nằm ở Tập 3 (Bảo mật — kiểm tra tham số deep link). Test của app mẫu mở thẳng `/tasks/t2`, `/tasks/khong-co` và `/abc` (404).

Web: nút Back của trình duyệt

Router tích hợp History API: mỗi lần điều hướng bằng Router là một mục lịch sử; nút Back của trình duyệt đi ngược theo thời gian (docs "Web support").

Lỗi và bẫy thường gặp

- `/tasks/new` **khớp nhầm** `:id` — đặt route cụ thể trước route có tham số.
- Dùng `context` sau `await` (sau `showDialog`) mà không kiểm tra `context.mounted`.
- Quên `refreshListenable` → đăng nhập xong mà `redirect` không chạy lại.
- **Redirect vòng lặp**: `redirect /login → /login...` luôn trả `null` khi đã ở đúng chỗ.
- **Open redirect**: dùng thẳng `from` từ URL để `go(from)` — kẻ xấu có thể gửi link ?
`from=https://...` Kiểm tra (Bài 1).
- **Pop khi stack trống** (mở từ deep link) → dùng `canPop()`.
- **Tạo GoRouter trong build** → mỗi lần build tạo router mới, mất trạng thái. Tạo một lần (trong `State`, hoặc biến top-level).

Tóm tắt

- `Navigator.push/pop` cho màn hình tạm, nhận kết quả bằng `await`.
- `go_router`: URL → màn hình; `go` đặt vị trí, `push` chồng lên; tham số qua `state.pathParameters / state.uri.queryParameters`.
- `StatefulShellRoute.indexedStack` cho tab giữ state; `errorBuilder` cho 404; `redirect + refreshListenable` cho guard.

Bài tập (có lời giải)

Bài 1. `LoginScreen` gọi `context.go(from ?? '/')` với `from` lấy từ URL. Viết `safeRedirectTarget(from)` chỉ cho phép đường dẫn **nội bộ** (bắt đầu bằng một `/`), chặn `https://evil.com`, `//evil.com`, `javascript:...`.

▼ Lời giải

`examples/lib/chapters/ch08/exercise_solution.dart`:

```
String safeRedirectTarget(String? from, {String fallback = '/'}) {
  if (from == null || from.isEmpty) return fallback;
  if (!from.startsWith('/') || from.startsWith('//')) return fallback;
  final uri = Uri.tryParse(from);
  if (uri == null || uri.hasScheme || uri.hasAuthority) return fallback;
  return from;
}
```

Rồi trong `LoginScreen`: `context.go(safeRedirectTarget(from))`. `//evil.com` là URL "protocol-relative" — trình duyệt hiểu là domain khác, nên phải chặn riêng. Test có 5 trường hợp (`ch08_test.dart`).

Bài 2. Trong app mẫu, vì sao nút "Lưu" ở form viết `if (context.canPop()) context.pop(); else context.go('/tasks');` thay vì chỉ `context.pop()` ?

▼ Lời giải

Form có thể được mở bằng **deep link** thẳng `/tasks/new`. Khi đó `go_router` vẫn dựng stack `/tasks` → `/tasks/new` (route con), nên `pop` thường được. Nhưng nếu route được cấu hình ở cấp cao nhất hoặc mở trong ngữ cảnh khác, stack chỉ có một trang và `pop()` sẽ báo lỗi "There is nothing to pop". `canPop()` giúp an toàn trong mọi trường hợp. Test "thêm việc: validate rồi lưu, quay về danh sách" và "chi tiết → sửa → lưu" (`app_test.dart`) chạy qua cả hai nhánh.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Navigation and routing — <https://docs.flutter.dev/ui/navigation> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/navigation/index.md>
- Tutorial: Stack based navigation — <https://docs.flutter.dev/learn/pathway/tutorial/navigation> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/navigation.md>
- Cookbook: Navigate to a new screen and back — <https://docs.flutter.dev/cookbook/navigation/navigation-basics> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/navigation/navigation-basics.md>
- Cookbook: Return data from a screen — <https://docs.flutter.dev/cookbook/navigation/returning-data> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/navigation/returning-data.md>
- Architecture recommendations (`go_router`) — <https://docs.flutter.dev/app-architecture/recommendations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/recommendations.md>
- Package `go_router` 18.0.2 (README trong pub cache sau `flutter pub get`): https://pub.dev/packages/go_router

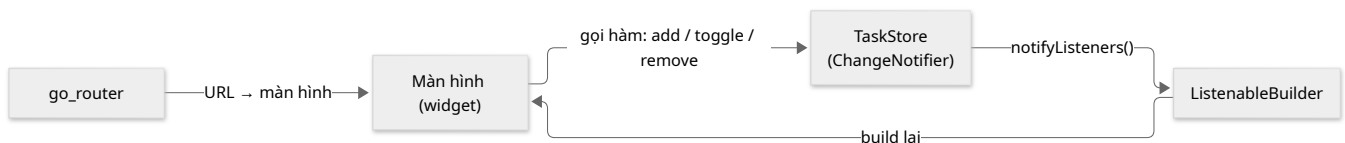
Chương 9 — App mẫu Tập 1: "Việc Cần Làm"

Mục tiêu

- Ghép mọi thứ của Tập 1 thành một app hoàn chỉnh: 3 tab, danh sách, form thêm/sửa, chi tiết, xóa có hoàn tác, lọc, dark mode, trang 404, deep link.
- Đọc cấu trúc thư mục của một app Flutter nhỏ.
- Chạy toàn bộ kiểm tra: `flutter analyze`, `flutter test`, `flutter build web`, `smoke test web`.

Giải thích đơn giản

App quản lý việc cần làm, dữ liệu trong bộ nhớ (Tập 2 sẽ lưu vào SQLite). Luồng dữ liệu một chiều:



- **Model** `Task`: bất biến, có `copyWith` và `==` theo giá trị.
- **Store** `TaskStore` extends `ChangeNotifier`: giữ danh sách, mọi thay đổi đi qua hàm của nó.
- **Màn hình**: đọc store qua `ListenableBuilder`, gọi hàm của store khi người dùng thao tác.
- **Router**: `StatefulShellRoute` 3 tab; store được truyền vào màn hình qua constructor (DI bằng tham số — Tập 2 thay bằng `provider`).

Ví dụ

Cấu trúc

<code>examples/lib/</code>	
<code>main.dart</code>	<code>runApp(TodoApp())</code>
<code>app.dart</code>	<code>TodoApp</code> : tạo store, <code>ThemeMode</code> (<code>ValueNotifier</code>), <code>GoRouter</code> ;
<code>MaterialApp.router</code>	
<code>theme.dart</code>	<code>AppTheme.light / dark</code> (<code>ColorScheme.fromSeed</code>)
<code>router.dart</code>	cây route: <code>/tasks</code> , <code>/tasks/new</code> , <code>/tasks/:id</code> , <code>/tasks/:id/edit</code> , <code>/lab</code> ,
<code>/lab/:chapter</code> , <code>/settings</code>	
<code>features/tasks/</code>	
<code>task.dart</code>	<code>Task</code> , <code>Priority</code> , <code>TaskFilter</code> , <code>sortTasks</code> , <code>filterTasks</code> , <code>searchTasks</code> ,
<code>validateTaskTitle</code>	
<code>task_store.dart</code>	<code>TaskStore</code> (<code>ChangeNotifier</code>)
<code>screens/</code>	<code>home_shell</code> , <code>task_list</code> , <code>task_detail</code> , <code>task_form</code> , <code>settings</code> , <code>not_found</code>
<code>lab/lab_screens.dart</code>	tab Lab: mở ví dụ từng chương
<code>chapters/ch01..ch08/</code>	ví dụ + lời giải bài tập
<code>examples/test/</code>	test từng chương + <code>features/task_test.dart</code> + <code>app_test.dart</code>

Store

```
class TaskStore extends ChangeNotifier {
  TaskStore([Iterable<Task> initial = const []]) : _tasks = [...initial] {
    _nextId = _tasks.length + 1;
  }

  final List<Task> _tasks;
  late int _nextId;

  /// Bản chỉ-đọc: bên ngoài không sửa trực tiếp được.
  List<Task> get tasks => List.unmodifiable(_tasks);

  int get remaining => _tasks.where((t) => !t.done).length;

  Task add({required String title, String note = '', Priority priority =
Priority.normal}) {
    final task = Task(id: 't${_nextId++}', title: title.trim(), note: note.trim(),
priority: priority);
    _tasks.add(task);
    notifyListeners();
    return task;
  }

  void toggle(String id) {
    final task = byId(id);
    if (task != null) update(task.copyWith(done: !task.done));
  }

  /// Xóa và trả về (vị trí, việc) để có thể "Hoàn tác".
  (int, Task)? remove(String id) {
    final i = _tasks.indexWhere((t) => t.id == id);
    if (i == -1) return null;
    final removed = _tasks.removeAt(i);
    notifyListeners();
    return (i, removed);
  }
  /// byId, update, insertAt, clearDone ... xem file đầy đủ
}
```

`remove` trả về **record** `(int, Task)?` — màn hình dùng nó cho nút "Hoàn tác" của `SnackBar`:

```
void _delete(Task task) {
  final removed = widget.store.remove(task.id);
  if (removed == null) return;
  final (index, oldTask) = removed;
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar()
    ..showSnackBar(SnackBar(
      content: Text('Đã xóa "${oldTask.title}"'),
      action: SnackBarAction(label: 'Hoàn tác', onPressed: () =>
widget.store.insertAt(index, oldTask)),
    ));
}
```

Màn hình danh sách đọc store

```
body: ListenableBuilder(  
  listenable: widget.store,  
  builder: (context, _) {  
    final visible = sortTasks(filterTasks(widget.store.tasks, _filter));  
    return Column(children: [ /* SegmentedButton lọc, "Còn N việc", ListView.builder +  
Dismissible */ ]);  
  },  
),
```

Bộ lọc `_filter` là **state cục bộ** của màn hình (`setState`), còn danh sách việc là **state của app** (store) — đúng phân loại "ephemeral vs app state" của docs.

Dark mode

`TodoApp` giữ `ValueNotifier<ThemeMode>`; tab Cài đặt đổi `themeMode.value` bằng `SegmentedButton`; `ValueListenableBuilder` bọc `MaterialApp.router` để đổi theme toàn app ngay lập tức.

Chạy

```
cd books/flutter/vol1-co-ban/examples  
flutter pub get  
flutter run -d chrome      # hoặc: flutter run (iPhone qua Mac + Xcode)
```

- Trên Chrome/web: đã build và smoke test trong sandbox (xem dưới).
- **Chạy trên iPhone/Android: NOT RUN** (không có Mac/Xcode/Android SDK trong sandbox).

Kết quả kiểm tra thật (2026-09-30)

Test của app (`flutter test -j 1 --reporter expanded test/features test/app_test.dart`):

```

00:00 +0: test/features/task_test.dart: Task (model thuần) copyWith tạo bản mới, bản cũ giữ nguyên
00:00 +1: test/features/task_test.dart: Task (model thuần) sortTasks: chưa xong trước, ưu tiên cao trước
00:00 +2: test/features/task_test.dart: Task (model thuần) filterTasks
00:00 +3: test/features/task_test.dart: Task (model thuần) validateTaskTitle
00:00 +4: test/features/task_test.dart: Bài 2: searchTasks không phân biệt dấu
00:00 +5: test/features/task_test.dart: TaskStore (ChangeNotifier) add / toggle / update / remove + insertAt báo cho listener
00:00 +6: test/features/task_test.dart: TaskStore (ChangeNotifier) Bài 1: clearDone xóa việc đã xong và báo listener một lần
00:00 +7: test/features/task_test.dart: TaskStore (ChangeNotifier) tasks là danh sách chỉ-đọc
00:00 +8: test/app_test.dart: mở app thấy danh sách + 3 tab
00:02 +9: test/app_test.dart: thêm việc: validate rồi lưu, quay về danh sách
00:03 +10: test/app_test.dart: đánh dấu xong, lọc "Đã xong"
00:03 +11: test/app_test.dart: chi tiết → sửa → lưu; xóa có hộp thoại xác nhận
00:04 +12: test/app_test.dart: nút "Xóa việc đã xong" trên AppBar
00:04 +13: test/app_test.dart: vuốt để xóa rồi Hoàn tác
00:04 +14: test/app_test.dart: deep link /tasks/t2 và /tasks/khong-co; route lạ → trang 404
00:04 +15: test/app_test.dart: Cài đặt: chuyển sang giao diện Tối
00:05 +16: test/app_test.dart: Lab mở được mọi chương
00:06 +17: All tests passed!

```

Toàn bộ dự án (bash books/flutter/scripts/check-all.sh vol1-co-ban, log đầy đủ: [../logs/check-all.txt](#)): format OK, flutter analyze → "No issues found!", flutter test → 58 test pass, flutter build web → "✓ Built build/web", smoke test trong Chromium headless (khung 390×844 như iPhone) → thấy "Việc cần làm", "Còn 3 việc chưa xong" → "WEB SMOKE OK".

Đi sâu

Vì sao test toàn app lại là widget test?

app_test.dart bơm (pump) cả TodoApp với router thật và store thật, rồi bấm như người dùng. Nó chạy trong vài giây, không cần thiết bị — đây là mức test docs khuyên có nhiều nhất ("Test architectural components separately, and together"). Integration test chạy trên thiết bị thật ở Tập 2, Chương 7.

Mẹo: mỗi lần pump một app mới trong cùng test phải dùng `key: UniqueKey()`, nếu không Flutter **tái sử dụng State cũ** (cùng loại widget, cùng vị trí) và router cũ vẫn được giữ — sách đã gặp đúng lỗi này (test deep link thứ hai không chuyển trang) và sửa bằng `UniqueKey()`.

Điều gì sẽ đổi ở Tập 2?

Tập 1 (đơn giản)	Tập 2 (theo khuyến nghị kiến trúc chính thức)
Store truyền qua constructor	provider (<code>ChangeNotifierProvider</code> , <code>context.watch/read</code>)
Dữ liệu trong bộ nhớ	Repository + SQLite (<code>sqlite</code>), cài đặt bằng <code>shared_preferences</code>
Một TaskStore cho mọi màn hình	ViewModel cho từng màn hình (MVVM)
Không có I/O	Future, trạng thái đang tải / lỗi

Lỗi và bẫy thường gặp

- **Sửa list trả về từ `store.tasks`** → `UnsupportedError` (cố ý: danh sách chỉ-đọc).
- **Quên `notifyListeners()` trong store** → UI không đổi.
- **Gọi `notifyListeners()` sau khi store đã `dispose`** → lỗi "A TaskStore was used after being disposed".
- **Pump lại app trong test không đổi key** → State/router cũ bị giữ.
- **Tạo router trong `build` của `TodoApp`** → mất lịch sử điều hướng mỗi lần đổi theme (router được tạo một lần bằng `late final`).

Tóm tắt

- App mẫu = model bất biến + store `ChangeNotifier` + `ListenableBuilder` + `go_router` 3 tab + theme sáng/tối.
- State cục bộ (bộ lọc) dùng `setState`; state app (danh sách) nằm trong store.
- Kiểm tra: analyze 0 issue, 58 test, build web, smoke test web — tất cả chạy thật.

Bài tập (có lời giải)

Bài 1. Thêm nút "Xóa việc đã xong" trên AppBar: xóa mọi việc `done`, hiện `SnackBar` "Đã xóa N việc đã xong". Store chỉ báo listener khi thật sự có thay đổi.

▼ Lời giải

`task_store.dart`:

```
int clearDone() {  
  final before = _tasks.length;  
  _tasks.removeWhere((t) => t.done);  
  final removed = before - _tasks.length;  
  if (removed > 0) notifyListeners();  
  return removed;  
}
```

```
task_list_screen.dart — AppBar(actions: [IconButton(tooltip: 'Xóa việc đã xong', onPressed:
() { final n = widget.store.clearDone();
ScaffoldMessenger.of(context)..hideCurrentSnackBar()..showSnackBar(...); }]]). Test đơn vị:
gọi 2 lần → lần 1 trả 1, lần 2 trả 0, listener được gọi đúng 1 lần. Test widget: bấm nút
→ thấy "Đã xóa 1 việc đã xong", còn 3 việc.
```

Bài 2. Viết `searchTasks(tasks, query)`: tìm theo tên hoặc ghi chú, không phân biệt hoa thường và **có/không dấu** ("hoc" tìm được "Học từ vựng"). Dùng lại extension ở Chương 3.

▼ Lời giải

task.dart:

```
List<Task> searchTasks(Iterable<Task> tasks, String query) {
  final q = query.trim().withoutAccents;
  if (q.isEmpty) return tasks.toList();
  return tasks.where((t) => t.title.withoutAccents.contains(q) ||
t.note.withoutAccents.contains(q)).toList();
}
```

(import '../..chapters/ch03/dart_basics.dart' show VietnameseText; — show chỉ nhập extension cần dùng.) Test: "hoc" → tìm thấy "Học từ vựng"; "DAM PHAN" → tìm thấy việc có ghi chú "Đàm phán hợp đồng"; chuỗi rỗng → trả tất cả. Nối vào UI: thêm một `SearchBox` (Chương 2) phía trên danh sách và gọi `searchTasks` trước `sortTasks`. App TOEIC (Task 9) dùng đúng kỹ thuật này cho ô tìm từ.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Tutorial: Use `ChangeNotifier` to update app state — <https://docs.flutter.dev/learn/pathway/tutorial/change-notifier> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/change-notifier.md>
- Tutorial: Use `ListenableBuilder` to update app UI — <https://docs.flutter.dev/learn/pathway/tutorial/listenable-builder> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/listenable-builder.md>
- Ephemeral vs app state — <https://docs.flutter.dev/data-and-backend/state-mgmt/ephemeral-vs-app> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4>

[a96238028f581a5/sites/docs/src/content/data-and-backend/state-mgmt/ephemeral-vs-app.md](https://github.com/flutter/flutter/blob/master/docs/src/content/data-and-backend/state-mgmt/ephemeral-vs-app.md)

- Architecture recommendations (testing, immutable models) — <https://docs.flutter.dev/app-architecture/recommendations> — <https://github.com/flutter/flutter/blob/master/docs/src/content/app-architecture/recommendations.md>
- Cookbook: Display a snackbar — <https://docs.flutter.dev/cookbook/design/snackbars> — <https://github.com/flutter/flutter/blob/master/docs/src/content/cookbook/design/snackbars.md>
- Package `go_router`: https://pub.dev/packages/go_router

STACK — Phiên bản và thư viện dùng cho cả bộ sách Flutter

Kiểm tra ngày **2026-09-30** bằng: file phát hành chính thức `releases_linux.json`, API của `pub.dev` (`https://pub.dev/api/packages/<tên>`), lệnh `flutter pub get` thật trong 3 dự án `vol*/examples/`, và **mã nguồn** tài liệu chính thức trên GitHub (`flutter/website`, `dart-lang/site-www`). Trang `docs.flutter.dev`, `api.flutter.dev`, `dart.dev` bị chặn trong sandbox, nên chúng tôi đọc file Markdown nguồn của các trang đó ở một commit cố định (xem "Nguồn tham khảo").

Task 9 (app TOEIC Flutter) phải dùng đúng bảng ở mục 5.

1. Tóm tắt nhanh (TL;DR)

Thành phần	Phiên bản ghim (pinned)	Vì sao
Flutter SDK	3.47.5 (kênh stable, phát hành 2026-09-18)	Bản stable mới nhất trong <code>releases_linux.json</code>
Dart SDK	3.13.4 (đi kèm Flutter 3.47.5)	Có sẵn trong Flutter; Dart 3.13 thêm <i>primary constructors</i>
Ràng buộc SDK trong <code>pubspec.yaml</code>	<code>sdk: ^3.13.4</code>	Mặc định của <code>flutter create</code> với SDK trên
State + DI	<code>provider 6.1.5+1</code> + <code>ChangeNotifier</code> (có sẵn trong SDK)	Docs chính thức khuyên dùng (xem mục 3)
Điều hướng	<code>go_router 18.0.2</code>	Docs chính thức khuyên dùng; do team Flutter duy trì
SQLite	<code>sqflite 2.4.4</code> (+ <code>sqflite_common_ffi 2.4.3</code> để test, <code>sqflite_common_ffi_web 1.2.0</code> cho web)	Cookbook chính thức dùng <code>sqflite</code>
Key-value	<code>shared_preferences 2.5.5</code>	Cookbook chính thức
Text-to-speech	<code>flutter_tts 4.2.5</code>	Docs không nêu tên — lựa chọn của sách
SQLite web	<code>sqlite3 3.6.0</code> (ghim, khớp file <code>wasm</code>)	Xem mục 5
Thông báo cục bộ	<code>flutter_local_notifications 22.3.1</code> + <code>timezone 0.11.1</code> + <code>flutter_timezone 5.1.0</code>	Docs không nêu tên — lựa chọn của sách
HTTP	<code>http 1.6.0</code>	Tutorial chính thức dùng package <code>http</code>
Test	<code>flutter_test</code> , <code>integration_test</code> (trong SDK), <code>mocktail 1.0.5</code>	Docs + case study chính thức
Lint	<code>flutter_lints 6.0.0</code>	Mặc định của <code>flutter create</code>
Bảo mật (Tập 3)	<code>flutter_secure_storage 11.2.0</code> , <code>crypto 3.0.7</code>	Docs không nêu tên — lựa chọn của sách

2. Flutter và Dart mới nhất

- `releases_linux.json` (2026-09-30): `current_release.stable` = 6a19cca... = **Flutter 3.47.5**, ngày 2026-09-18, Dart **3.13.4**. Kênh beta đang ở 3.49.0-0.1.pre (Dart 3.14.0) — **không dùng beta**.
- Trong sandbox: `flutter --version` → Flutter 3.47.5 • channel stable • Framework revision 6a19cca564 • Tools • Dart 3.13.4 • DevTools 2.60.0.
- Flutter 3.47 ra mắt 2026-08-12 (trang "What's new" của docs). Điểm đáng chú ý cho sách: Widget Previews lên stable; hướng dẫn Swift Package Manager mở rộng.
- Từ Flutter 3.44, **Swift Package Manager (SwiftPM)** là mặc định để quản lý phụ thuộc native iOS/macOS; CocoaPods ở chế độ bảo trì (maintenance mode), và registry CocoaPods sẽ chỉ đọc từ 2026-12-02 (docs `packages-and-plugins/swift-package-manager/for-app-developers`).
- Dart 3.13.0 (2026-08-12, CHANGELOG của Dart SDK): thêm **primary constructors** (`class Point(var int x, var int y);`) — rất giống "parameter properties" của TypeScript. Breaking change nhỏ: không được viết `final/var` trước tham số thường nữa. Dart 3.12 thêm private named parameters. Sách dùng cú pháp cổ điển là chính và giới thiệu primary constructors trong Tập 1, Chương 3 (đã compile thật).

3. Docs chính thức khuyên gì? (state, DI, routing, dữ liệu, test)

Trang **Architecture recommendations** (`app-architecture/recommendations`, dữ liệu trong `src/data/architectureRecommendations.yml`) xếp hạng khuyến nghị:

Khuyến nghị của docs	Mức độ	Sách áp dụng thế nào
Tách rõ data layer và UI layer	strong	Tập 2–3: <code>data/</code> (repository, service) và <code>ui/</code> (view, view model)
Repository pattern ở data layer	strong	<code>WordRepository</code> , <code>NoteRepository</code> (abstract + bản SQLite + bản giả để test)
MVVM: View + ViewModel ở UI layer	strong	<code>ViewModel</code> = <code>ChangeNotifier</code> ; View = widget "ngốc"
Dùng <code>ChangeNotifier</code> + <code>Listenable</code> để cập nhật widget	conditional	Dùng; so sánh Riverpod, Bloc, signals ở Tập 2 Ch.1
DI bằng package provider	strong	<code>MultiProvider</code> ở gốc app
Điều hướng bằng go_router ("preferred way to write 90% of Flutter applications")	recommend	Cả 3 tập
Command pattern cho sự kiện người dùng	recommend	Tập 3 (<code>Command0/Command1</code>) và app mẫu Tập 2 dạng đơn giản
Model bất biến (immutable)	strong	<code>final field</code> + <code>copyWith</code> viết tay (không dùng <code>freezed</code> để tránh code generation)
Test từng thành phần + dùng fake	strong	Mỗi repository có bản <code>Fake...</code> ; <code>mocktail</code> chỉ khi cần kiểm tra lời gọi

Thêm:

- Trang `ui/navigation` không khuyên dùng `named routes` cho đa số app; khuyên dùng `go_router`.
- Trang `data-and-backend/state-mgmt/simple` dạy `provider`; Learning Pathway dạy `ChangeNotifier` + `ListenableBuilder` (không cần package).
- Cookbook `persistence/sqlite` dùng `sqflite` + `path`; ví dụ kiến trúc chính thức `examples/app-architecture/todo_data_service` dùng `sqflite`, `sqflite_common_ffi`, `shared_preferences`.
- Case study kiến trúc (`app-architecture/case-study/testing`) mock router bằng `package:mocktail`; cookbook `testing/unit/mocking` dùng `mockito` (cần `build_runner` sinh code). **Quyết định:** fake viết tay là chính, `mocktail` (không sinh code) khi cần `verify`.

So sánh thư viện state (Tập 2)

Repo	Stars	Cập nhật gần nhất (updated_at)	License	Hữu ích	Ngày kiểm tra
https://github.com/rrousselGit/provider	5.2k	2026-09-29	MIT	Docs chính thức khuyên; giống DI của Angular	2026-09-30
https://github.com/rrousselGit/riverpod	7.4k	2026-09-29	MIT	Mạnh, an toàn compile-time, cache async	2026-09-30
https://github.com/felangel/bloc	12.5k	2026-09-30	MIT	Luồng sự kiện → state, giống NgRx	2026-09-30
https://github.com/rodydavis/signals.dart	0.8k	2026-09-22	Apache-2.0	API giống Angular Signals	2026-09-30

Số liệu pub.dev (2026-09-30, API `/score`): `provider` 11,008 likes; `flutter_riverpod` 2,910 likes (pub.dev không gắn tag web cho `flutter_riverpod` 3.4.3); `flutter_bloc` 8,081 likes; `signals` 710 likes.

Quyết định: `provider` + `ChangeNotifier`. Lý do: (1) docs chính thức khuyên; (2) `ChangeNotifier` nằm trong SDK, ít thứ phải học; (3) map thẳng sang `service` + DI của Angular; (4) một agent khác (Task 9) sẽ làm theo sách — chọn con đường chính thức ít bất ngờ nhất.

4. Chạy app Flutter trên iPhone — nói thật

Không có "Expo Go" cho Flutter. Flutter biên dịch Dart thành mã máy (AOT) cho iOS, nên muốn chạy app native trên iPhone phải **build bằng Xcode trên máy Mac**. Các lựa chọn thật của Nobin:

Cách	Cần gì	Ưu / nhược
A. Mac + Xcode + cáp (flutter run)	Máy Mac, Xcode, Apple ID miễn phí	Cách chính thức. Hot reload trên máy thật. Xem các bước dưới
B. Web build trên Safari (flutter run -d web-server hoặc flutter build web)	Chỉ cần máy tính + iPhone cùng Wi-Fi	Không cần Mac. Nhưng là web , không phải app native: không có thông báo cục bộ theo lịch; SQLite chạy bằng WebAssembly
C. TestFlight	Mac (hoặc Mac trên cloud CI), Apple Developer Program trả phí	Cài qua app TestFlight, giống app thật; cần build + upload

Cách A — các bước theo docs chính thức (platform-integration/ios/setup):

1. Cài Xcode, chạy `sudo sh -c 'xcode-select -s /Applications/Xcode.app/Contents/Developer && xcodebuild -runFirstLaunch', sudo xcodebuild -license, xcodebuild -downloadPlatform iOS.`
 2. (Tùy chọn) CocoaPods cho plugin chưa hỗ trợ SwiftPM.
 3. Cắm iPhone vào Mac → bấm **Trust**.
 4. Bật **Developer Mode**: Settings → Privacy & Security → Developer Mode → On → khởi động lại.
 5. Tạo chứng chỉ ký (signing) bằng Apple ID; docs viết: tài khoản developer cá nhân miễn phí dùng được khi chỉ *test* trên máy của mình.
 6. Trên iPhone: Settings → VPN & Device Management → tin cậy (Trust) chứng chỉ developer.
 7. `cd books/flutter/vol1-co-ban/examples && flutter run` (chọn iPhone).
- **UNVERIFIED**: app ký bằng tài khoản miễn phí hết hạn sau 7 ngày và phải cài lại. Đây là điều cộng đồng hay nói; chúng tôi không mở được trang Apple nào xác nhận con số này.
 - Trang Apple (đã mở 2026-09-30, <https://developer.apple.com/programs/>): đăng ký tài khoản miễn phí cho phép chạy app trên thiết bị của mình bằng Xcode; **Apple Developer Program: \$99 mỗi năm**, cần cho TestFlight và App Store.

Cách B — web trên Safari (không cần Mac):

```
cd books/flutter/vol2-trung-cap/examples
flutter build web --no-web-resources-cdn --release
cd build/web && python3 -m http.server 8080 --bind 0.0.0.0
# trên iPhone (cùng Wi-Fi): mở Safari → http://<IP-máy-tính>:8080
```

- Docs (platform-integration/web/faq): Flutter web chạy trên Safari (mobile & desktop).
- Docs (platform-integration/web/wasm): bản build **Wasm** không chạy trên bất kỳ trình duyệt nào của iOS → **không** dùng `--wasm` cho iPhone; bản JavaScript mặc định thì được.

- `--no-web-resources-cdn`: dùng CanvasKit đóng gói sẵn thay vì tải từ CDN (sandbox chặn CDN; trên iPhone cũng giúp chạy khi mạng chậm).
- **Chạy trên iPhone: NOT RUN** (không có iPhone/Mac trong sandbox). Bản web đã được kiểm tra trong Chromium headless (xem `logs/`). Hành vi trên Safari iOS (TTS bằng Web Speech, SQLite Wasm) là **UNVERIFIED**.

Cách C — Mac trên cloud: GitHub Actions có runner macOS; Codemagic cũng build iOS. Vẫn cần Apple Developer Program để ký và phân phối. Chi tiết ở Tập 3 Ch.5–6. Giá/điều khoản dịch vụ CI: **UNVERIFIED** (trang docs.github.com và codemagic.io bị chặn trong sandbox).

5. Thư viện cho từng tập (bảng cho Task 9)

Tất cả ghim **exact** (không `^`) trong `pubspec.yaml`; `pubspec.lock` được commit. Đã chạy `flutter pub get` thành công với Flutter 3.47.5 / Dart 3.13.4 trong các dự án `vol*/examples`.

Tập	Package	Phiên bản	Publisher (pub.dev)	License	Nền tảng (tag pub.dev)	Docs nêu tên?	D
1	go_router	18.0.2	flutter.dev	BSD-3-Clause	android, ios, web, ...	Có	Điều hướng link, tài liệu (ShellRoute)
2	shared_preferences	2.5.5	flutter.dev	BSD-3-Clause	android, ios, web, ...	Có	Lưu cài đặt cục bộ
1–3	flutter_lints (dev)	6.0.0	flutter.dev	BSD-3-Clause	tất cả	Có (mặc định)	Lint
2	provider	6.1.5+1	dash-overflow.net	MIT	android, ios, web, ...	Có	DI + lắng nghe thay đổi
2	sqflite	2.4.4	tekartik.com	BSD-2-Clause	android, ios, macos	Có	SQLite cơ sở dữ liệu
2	path	1.9.1	dart.dev	BSD-3-Clause	tất cả	Có	Ghép đường dẫn file DB
2	sqflite_common_ffi (dev)	2.4.3	tekartik.com	BSD-2-Clause	tất cả	Có (trong ví dụ kiến trúc)	Chạy SQLite trong môi trường test
2	sqflite_common_ffi_web	1.2.0	tekartik.com	BSD-2-Clause	web	Có (docs gọi là <i>experimental</i>)	SQLite bản web
2	sqlite3	3.6.0	simonbinder.eu	MIT	tất cả	Không (phụ thuộc gián tiếp)	Ghim dữ liệu web/sc (bản sqlite3)
2	http	1.6.0	dart.dev	BSD-3-Clause	tất cả	Có	Gọi API
2	flutter_tts	4.2.5	eyedeadevelopment.com	MIT	android, ios, web, ...	Không	Đọc to (text-to-speech)
2	flutter_local_notifications	22.3.1	dexterx.dev	BSD-3-Clause	android, ios, ...	Không	Nhắc nhở ngay
2	timezone	0.11.1	labs.dart.dev	BSD-2-Clause	tất cả	Không	Lịch thời gian (zoned)
2	flutter_timezone	5.1.0	wolverinebeach.net	Apache-2.0	android, ios, web, ...	Không	Lấy múi giờ máy (Intl)
2	mocktail (dev)	1.0.5	felangel.dev	MIT	tất cả	Có (case study)	Mock kiểm tra verify
2	integration_test (dev)	SDK	flutter.dev	BSD-3-Clause	—	Có	Integration test

Tập	Package	Phiên bản	Publisher (pub.dev)	License	Nền tảng (tag pub.dev)	Docs nêu tên?	D
3	flutter_secure_storage	11.2.0	steenbakker.dev	BSD-3-Clause	android, ios, web, ...	Không	Lưu bí (Keychain)
3	crypto	3.0.7	dart.dev	BSD-3-Clause	tất cả	Không	Băm PBKDF2 + salt)

Không dùng (và lý do):

- `drift` 2.35.0 (type-safe SQL, reactive) — mạnh nhưng cần code generation (`build_runner`); `sqflite` đủ cho app TOEIC và là lựa chọn của cookbook. Ghi ở "Ideas for later".
- `hive` 2.2.3, `isar` 3.1.0+1 — bản mới nhất từ 2022/2023, ràng buộc SDK `<3.0.0` → **không** dùng được với Dart 3.13.
- `sqlite3_flutter_libs` — bản mới nhất là `0.6.0+eol` (end of life).
- `freezed/json_serializable` — docs "recommend" `freezed`, nhưng sách viết `fromJson/copyWith` bằng tay để người mới thấy rõ cơ chế và để build không cần `build_runner`. Tập 2 Ch.3 giới thiệu `json_serializable`.

Về việc ghim phiên bản: trang Security của docs khuyên **tránh ghim cứng** và nếu ghim thì phải kiểm tra bản vá định kỳ. Sách ghim exact để mọi người (và Task 9) build ra cùng kết quả; đổi lại, hãy chạy `flutter pub outdated` hằng tháng và nâng phiên bản có chủ đích (chạy lại `scripts/check-all.sh` sau khi nâng).

Ghi chú nền tảng quan trọng cho Task 9:

- `sqflite` không có bản web. Trên web phải gán `databaseFactory = databaseFactoryFfiWeb` và có 2 file `web/sqlite3.wasm` + `web/sqflite_sw.js` (tạo bằng `dart run sqflite_common_ffi_web:setup`, đã commit trong dự án Tập 2).
- `flutter_local_notifications` 22 cần bật *core library desugaring* trong Gradle của Android (README của package). Trên iOS phải xin quyền (`requestPermissions`). Web không có lịch thông báo.
- `flutter_tts` trên iOS: dùng `setIosAudioCategory` nếu muốn đọc cả khi bật chế độ im lặng (README).

6. Công cụ trong sandbox (để kiểm tra sách)

Công cụ	Phiên bản
Flutter / Dart / DevTools	3.47.5 / 3.13.4 / 2.60.0 (tại <code>/opt/flutter</code>)
Node / npm	22.22.2 / 10.9.7
pandoc	3.1.3
playwright-core / mermaid (build PDF, smoke test web)	1.56.1 / 12.0.0
Chromium (Playwright)	<code>/opt/pw-browsers/chromium-1194</code>

Không có: Xcode, Android SDK, thiết bị thật, máy ảo. Mọi mục "chạy trên iPhone/Android" là **NOT RUN**.

7. Bản quyền nội dung tham khảo

- Repo `flutter/website` (nguồn của `docs.flutter.dev`): "Except as otherwise noted", nội dung theo **Creative Commons Attribution 3.0**, code mẫu theo **BSD License** (file `LICENSE` của repo, kiểm tra 2026-09-30).
- Repo `dart-lang/site-www` (nguồn của `dart.dev`): nội dung **CC BY 4.0**, code mẫu **BSD-3-Clause** (file `LICENSE`).
- Sách viết lại bằng lời của mình (paraphrase), chỉ trích ngắn, luôn ghi nguồn. Code trong `examples/` do sách tự viết.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30:

- Flutter releases (Linux): https://storage.googleapis.com/flutter_infra_release/releases/releases_linux.json
- Dart SDK CHANGELOG (3.12, 3.13): <https://github.com/dart-lang/sdk/blob/main/CHANGELOG.md>
- Docs Flutter — mã nguồn tại commit `ab59c614e780e2d6d44f07ae4a96238028f581a5` (trang tương ứng trên `docs.flutter.dev` bị chặn trong sandbox):
 - Architecture recommendations — <https://docs.flutter.dev/app-architecture/recommendations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/recommendations.md> và <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/data/architectureRecommendations.yml>
 - Navigation and routing — <https://docs.flutter.dev/ui/navigation> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/navigation/index.md>

- Simple app state management — <https://docs.flutter.dev/data-and-backend/state-mgmt/simple> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/data-and-backend/state-mgmt/simple.md>
- State management options — <https://docs.flutter.dev/data-and-backend/state-mgmt/options> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/data-and-backend/state-mgmt/options.md>
- Persist data with SQLite — <https://docs.flutter.dev/cookbook/persistence/sqlite> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/persistence/sqlite.md>
- Ví dụ kiến trúc todo_data_service (pubspec): https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/examples/app-architecture/todo_data_service/pubspec.yaml
- Case study — testing — <https://docs.flutter.dev/app-architecture/case-study/testing> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/case-study/testing.md>
- Set up iOS development — <https://docs.flutter.dev/platform-integration/ios/setup> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/platform-integration/ios/setup.md>
- Swift Package Manager for app developers — <https://docs.flutter.dev/packages-and-plugins/swift-package-manager/for-app-developers> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/packages-and-plugins/swift-package-manager/for-app-developers.md>
- Web FAQ — <https://docs.flutter.dev/platform-integration/web/faq> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/platform-integration/web/faq.md>
- Wasm — <https://docs.flutter.dev/platform-integration/web/wasm> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/platform-integration/web/wasm.md>
- What's new — <https://docs.flutter.dev/release/whats-new> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/release/whats-new.md>
- LICENSE của repo docs: <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/LICENSE>
- Dart docs — LICENSE: <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/LICENSE>
- pub.dev (phiên bản, publisher, license, nền tảng): <https://pub.dev/packages/provider>, https://pub.dev/packages/go_router, <https://pub.dev/packages/sqlite>, https://pub.dev/packages/sqlite_common_ffi, https://pub.dev/packages/sqlite_common_ffi_web, <http://pub.dev/packages/http>

[s://pub.dev/packages/shared_preferences](https://pub.dev/packages/shared_preferences), <https://pub.dev/packages/http>, <https://pub.dev/packages/path>, https://pub.dev/packages/flutter_tts, https://pub.dev/packages/flutter_local_notifications, <https://pub.dev/packages/timezone>, https://pub.dev/packages/flutter_timezone, <https://pub.dev/packages/mocktail>, https://pub.dev/packages/flutter_lints, https://pub.dev/packages/flutter_secure_storage, <https://pub.dev/packages/crypto>, https://pub.dev/packages/flutter_riverpod, https://pub.dev/packages/flutter_bloc, <https://pub.dev/packages/signals>, <https://pub.dev/packages/drift>, <https://pub.dev/packages/hive>, <https://pub.dev/packages/isar>, https://pub.dev/packages/sqlite3_flutter_libs, <https://pub.dev/packages/sqlite3>

- README của package (đọc trong pub cache sau `flutter pub get`):
`flutter_local_notifications` 22.3.1, `flutter_tts` 4.2.5.
- Apple Developer Program: <https://developer.apple.com/programs/>

GLOSSARY — Thuật ngữ Flutter (Anh → Việt)

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular / React Native
Flutter	Bộ công cụ UI của Google: một codebase Dart → app iOS, Android, web, desktop; tự vẽ mọi pixel	Angular (web) / React Native (mobile)
Dart	Ngôn ngữ lập trình của Flutter: có kiểu, null safety, biên dịch AOT/JIT/JS/Wasm	TypeScript
Widget	Mô tả bất biến (immutable) của một phần giao diện; "mọi thứ là widget"	Component
Widget tree (cây widget)	Các widget lồng nhau tạo thành cây	Cây component / DOM
StatelessWidget	Widget không có state riêng; chỉ phụ thuộc tham số	Component "dumb" / presentational
StatefulWidget	Widget có object <code>State</code> sống lâu, đổi bằng <code>setState</code>	Component có state
State (object)	Nơi giữ dữ liệu thay đổi và các hàm vòng đời <code>initState</code> , <code>dispose</code>	Class component + lifecycle hooks
<code>build()</code>	Hàm trả về cây widget ứng với state hiện tại	Template / render
<code>setState()</code>	Báo Flutter state đã đổi → build lại widget	<code>signal.set()</code> / <code>setState</code> của React
BuildContext	"Vị trí" của widget trong cây; dùng để tìm Theme, Navigator, provider ở trên	Injector hiện tại
Element	Đối tượng Flutter tạo từ widget để quản lý cây thật (ít khi dùng trực tiếp)	—
RenderObject	Đối tượng làm layout và vẽ (tầng thấp)	—
Key	Định danh giúp Flutter giữ đúng state khi danh sách đổi thứ tự	<code>track</code> trong <code>@for</code> / <code>key</code> của React
Constraints (ràng buộc)	Cha nói cho con kích thước tối thiểu/tối đa; con chọn kích thước trong đó	— (khác CSS)
Row / Column / Flex	Bố cục ngang / dọc	Flexbox <code>flex-direction</code>
Expanded / Flexible	Con chiếm phần còn lại của Row/Column	<code>flex: 1</code>
Material 3	Hệ thiết kế của Google, mặc định trong Flutter	Angular Material
Cupertino	Bộ widget kiểu iOS	—
ThemeData / ColorScheme	Cấu hình màu, chữ toàn app; hỗ trợ sáng/tối	Theme SCSS / CSS variables
Sliver	Mảnh của vùng cuộn (SliverList, SliverAppBar) cho hiệu ứng cuộn tùy biến	Virtual scroll (CDK)
Navigator	Ngăn xếp (stack) màn hình kiểu mệnh lệnh: <code>push</code> / <code>pop</code>	<code>router.navigate</code> + history
go_router	Package định tuyến theo URL (declarative), docs chính thức khuyên dùng	Angular Router
Deep link	Mở app thẳng vào một màn hình bằng URL	Route URL
Redirect (go_router)	Hàm chuyển hướng trước khi hiển thị route	Route guard <code>canActivate</code>

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular / React Native
ShellRoute / StatefulShellRoute	Route "khung" bọc các route con (tab bar)	Layout component + <code><router-outlet></code>
Future	Một giá trị sẽ có trong tương lai	Promise
Stream	Chuỗi nhiều giá trị theo thời gian	Observable (RxJS)
<code>async</code> / <code>await</code>	Viết code bất đồng bộ như đồng bộ	Giống TypeScript
Isolate	"Luồng" riêng có bộ nhớ riêng để chạy việc nặng	Web Worker
ChangeNotifier	Class có sẵn, gọi <code>notifyListeners()</code> để báo đổi	Service có state + Subject/Signal
ValueNotifier	ChangeNotifier giữ một giá trị	<code>signal()</code>
ListenableBuilder / ValueListenableBuilder	Widget build lại khi Listenable báo đổi	<code>async pipe</code> / đọc signal trong template
InheritedWidget	Cơ chế truyền dữ liệu xuống cây, con tìm lên bằng context	Hierarchical injector
provider	Package DI + lắng nghe ChangeNotifier, docs chính thức khuyên dùng	Angular DI (<code>providers</code> , <code>inject</code>)
MVVM	Model–View–ViewModel: View = widget, ViewModel = ChangeNotifier	Component + service/facade
Repository	Lớp che giấu nguồn dữ liệu (SQLite, API) khỏi UI	Data service
Command (pattern)	Đối tượng bọc một hành động bất đồng bộ + trạng thái chạy/lỗi	NgRx effect đơn giản
Hot reload	Nạp lại code khi đang chạy, giữ state (phím <code>r</code>)	HMR / Fast Refresh
Hot restart	Khởi động lại app, mất state (phím <code>R</code>)	Tải lại trang
pub / pub.dev	Trình quản lý package và kho package của Dart	npm / npmjs.com
<code>pubspec.yaml</code> / <code>pubspec.lock</code>	Khai báo phụ thuộc / khóa phiên bản	<code>package.json</code> / <code>package-lock.json</code>
Plugin	Package có code native (Swift/Kotlin)	Native module (RN)
Platform channel (MethodChannel)	Kênh gọi qua lại giữa Dart và code native	TurboModule / bridge (RN)
Pigeon	Công cụ sinh code type-safe cho platform channel	Codegen (RN)
Swift Package Manager (SwiftPM)	Trình quản lý phụ thuộc iOS, mặc định từ Flutter 3.44	CocoaPods
AOT / JIT	Biên dịch trước (bản release) / biên dịch lúc chạy (debug, hot reload)	<code>ng build</code> / <code>ng serve</code>
Debug / Profile / Release mode	3 chế độ build: gỡ lỗi / đo hiệu năng / phát hành	dev / prod build
DevTools	Bộ công cụ gỡ lỗi, đo hiệu năng của Flutter	Chrome DevTools, Angular DevTools
Impeller	Engine vẽ mới của Flutter (mặc định trên iOS)	—

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular / React Native
CanvasKit / skwasm	Bộ vẽ của Flutter web (Skia biên dịch sang WebAssembly)	—
Semantics	Cây thông tin trợ năng (VoiceOver đọc); test cũng tìm theo semantics	ARIA
Widget test	Test một widget trong môi trường giả, không cần thiết bị	Component test (TestBed)
Integration test	Test cả app trên thiết bị/giả lập (package integration_test)	E2E (Playwright/Cypress)
Fake / Mock	Bản giả tự viết (fake) / đối tượng giả kiểm tra lời gọi (mock, mocktail)	<code>jasmine.createSpyObj</code>
sqflite	Plugin SQLite cho iOS/Android	expo-sqlite
shared_preferences	Lưu cặp key-value nhỏ	localStorage / AsyncStorage
TTS (text-to-speech)	Đọc chữ thành giọng nói (flutter_tts)	expo-speech
Local notification	Thông báo do app tự đặt lịch, không cần server	expo-notifications (local)
Keychain / Keystore	Kho bí mật của iOS / Android (flutter_secure_storage)	expo-secure-store
Obfuscation	Làm rối tên hàm/lớp trong bản build để khó dịch ngược	Minify
Flavor	Biến thể build (dev/staging/prod)	Angular environments
TestFlight	Kênh phát bản thử iOS của Apple	—